

Super-Human Performance in Gran Turismo Sport Using Deep Reinforcement Learning

Florian Fuchs¹, Yunlong Song², Elia Kaufmann², Davide Scaramuzza², and Peter Dürri¹

Abstract—Autonomous car racing is a major challenge in robotics. It raises fundamental problems for classical approaches such as planning minimum-time trajectories under uncertain dynamics and controlling the car at the limits of its handling. Besides, the requirement of minimizing the lap time, which is a sparse objective, and the difficulty of collecting training data from human experts have also hindered researchers from directly applying learning-based approaches to solve the problem. In the present work, we propose a learning-based system for autonomous car racing by leveraging a high-fidelity physical car simulation, a course-progress proxy reward, and deep reinforcement learning. We deploy our system in Gran Turismo Sport, a world-leading car simulator known for its realistic physics simulation of different race cars and tracks, which is even used to recruit human race car drivers. Our trained policy achieves autonomous racing performance that goes beyond what had been achieved so far by the built-in AI, and, at the same time, outperforms the fastest driver in a dataset of over 50,000 human players.

Index Terms—Autonomous Agents, Reinforcement Learning

SUPPLEMENTARY VIDEOS

This paper is accompanied by a narrated video of the performance: <https://youtu.be/Zeyv1bN9v4A>

I. INTRODUCTION

THE goal of autonomous car racing is to complete a given track as fast as possible, which requires the agent to generate fast and precise actions even when the vehicle reaches its physical limits. Classical approaches [1]–[3] generally decouple the racing problem into trajectory planning and vehicle control. From the trajectory planning standpoint, the problem is about how to compute the minimum-time trajectory for a given race track. From the vehicle control standpoint, it captures the main difficulty and the most critical scenario in autonomous driving—how to safely control the vehicle during very extreme manoeuvres.

Manuscript received: October 15, 2020; Revised January 9, 2021; Accepted February 12, 2021.

This paper was recommended for publication by Markus Vincze upon evaluation of the Associate Editor and Reviewers' comments. This work was supported by Sony R&D Center Europe Stuttgart Laboratory 1, the National Centre of Competence in Research (NCCR) Robotics through the Swiss National Science Foundation, and the SNSF-ERC Starting Grant.

¹F. Fuchs and P. Dürri are with Sony Europe B.V., Schlieren/Switzerland Branch (Florian.Fuchs@sony.com)

²Y. Song, E. Kaufmann, D. Scaramuzza are with the Robotics and Perception Group, University of Zurich, Switzerland (<http://rpg.ifi.uzh.ch>)

Digital Object Identifier 10.1109/LRA.2021.3064284

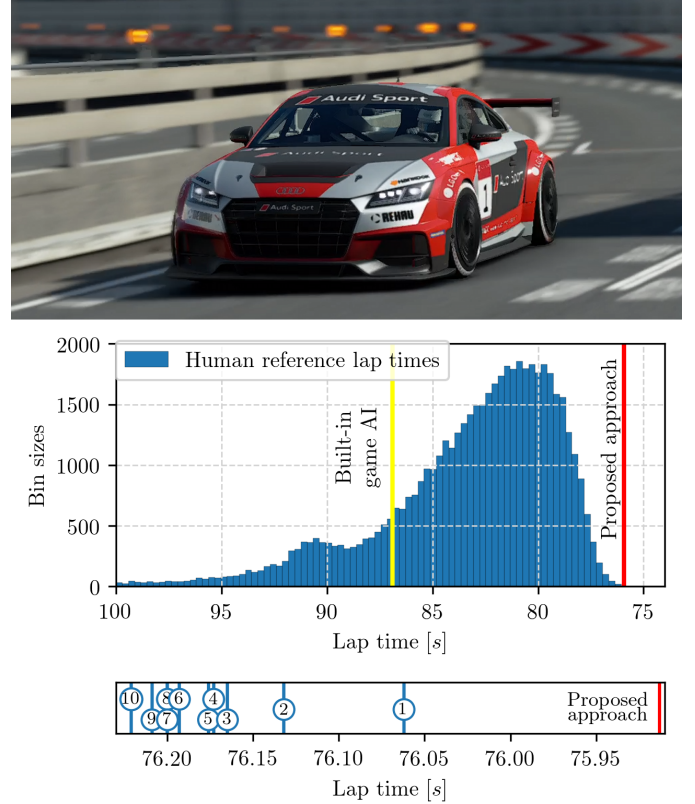


Fig. 1. **Top:** Our approach controlling the “Audi TT Cup” in the *Gran Turismo Sport* simulation. **Center:** Time trial lap time comparison between our approach (red line), the personal best lap times of over 50,000 human drivers from 70 countries (dark blue histogram), as well as the built-in non-player character (“game AI”, yellow line). Human lap times over 100 seconds are cut off to simplify the visualization. **Bottom:** Closer view at the lap times of the 10 fastest human drivers and our approach. **Note:** Starting with GTS update 1.57 the simulation dynamics changed such that the achievable times for the tested settings are different. See appendix VI-H for updated results under the new dynamics.

Classical approaches have been widely studied for addressing the autonomous car racing problem and have shown impressive results. In particular, much of the success relies on optimization-based techniques for trajectory generation and tracking, thanks to their capabilities of handling different constraints and nonlinear dynamics. However, several limitations exist in this line of research, such as the requirement of expensive computations for the nonlinear optimization and the lack of flexibility in the objective (most optimizations require quadratic cost formulations). Overall, conventional systems for autonomous driving escalate in complexity as the model complexity increases and do not leverage the large volume of

利用深度强化学习在 Gran Turismo 运动中实现超人表现

弗洛里安·福克斯¹、宋云龙²、埃利亚·考夫曼²、大卫·斯卡拉穆扎²和彼得·Dürr¹

Abstract—自动赛车是机器人技术的一项重大挑战。它提出了经典方法的基本问题，例如在不确定的动态下规划最短时间轨迹以及在其操控极限下控制汽车。此外，最小化单圈时间的要求（这是一个稀疏的目标）以及从人类专家那里收集训练数据的困难也阻碍了研究人员直接应用基于学习的方法来解决。在目前的工作中，我们通过利用高保真物理汽车模拟、课程进度代理奖励和深度强化学习，提出了一种基于学习的自动赛车系统。我们在 Gran Turismo Sport 中部署了我们的系统，这是一款世界领先的汽车模拟器，以其对不同赛车和赛道的真实物理模拟而闻名，甚至用于招募人类赛车手。我们训练有素的策略实现了自动赛车性能，超越了内置人工智能迄今为止所取得的成绩，同时，在超过 50,000 名人类玩家的数据集中，表现优于最快的驾驶员。

Index Terms——自主代理，强化学习

补充视频

本文附有表演的解说视频: <https://youtu.be/Zeyv1bN9v4>

一、简介

自动赛车的目标是尽快完成给定的赛道，这要求智能体即使在车辆达到其物理极限时也能产生快速而精确的动作。经典方法 [1]-[3] 通常将赛车问题解耦为轨迹规划和车辆控制。从轨迹规划的角度来看，问题是如何计算给定赛道的最短时间轨迹。从车辆控制的角度来看，它抓住了自动驾驶中的主要困难和最关键的场景——如何在非常极端的操纵过程中安全地控制车辆。

稿件收到时间: 2020年10月15日; 2021 年 1 月 9 日修订; 2021 年 2 月 12 日接受。

根据副主编和审稿人的意见, Markus Vincze 建议发表本文。这项工作得到了索尼欧洲研发中心斯图加特实验室 1、瑞士国家科学基金会国家机器人研究能力中心 (NCCR) 以及 SNSF-ERC 启动基金的支持。

1F. Fuchs 和 P. Dürr 就职于 Sony Europe B.V. Schlieren/瑞士分公司 (Florian.Fuchs@sony.com)

2Y. Song, E. Kaufmann, D. Scaramuzza 就职于瑞士苏黎世大学机器人与感知小组 (<http://rpg.ifi.uzh.ch>)

数字对象标识符 10.1109/LRA.2021.3064284

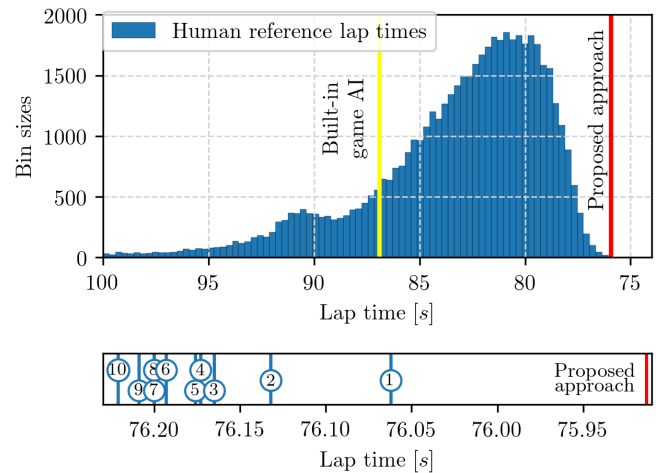


图 1. 顶部: 我们在 Gran Turismo Sport 模拟中控制“奥迪 TT Cup”的方法。中: 我们的方法 (红线)、来自 70 个国家的 50,000 多名人类驾驶员的个人最佳单圈时间 (深蓝色直方图) 以及内置非玩家角色 (“游戏 AI”, 黄线) 之间的计时赛单圈时间比较。为了简化可视化, 超过 100 秒的人类单圈时间被删除。底部: 近距离观察 10 名最快的人类车手的单圈时间和我们的方法。注意: 从 GTS 更新 1.57 开始, 模拟动态发生了变化, 因此测试设置的可实现时间有所不同。有关新动态下的更新结果, 请参阅附录 VI-H。

经典方法已被广泛研究来解决自动驾驶赛车问题, 并已显示出令人印象深刻的结果。特别是, 大部分成功依赖于基于优化的轨迹生成和跟踪技术, 这要归功于它们处理不同约束和非线性动力学的能力。然而, 这一研究领域存在一些局限性, 例如非线性优化需要昂贵的计算以及目标缺乏灵活性 (大多数优化需要二次成本公式)。总体而言, 传统的自动驾驶系统的复杂性随着模型复杂性的增加而增加, 并且没有利用大量的数据

data generated by real-world driving.

The requirements of improving vehicle models using real-world data and handling more complex cost formulations and general nonlinear dynamics motivated researchers to design learning-based systems. The first is related to dynamic model identification. In particular, multi-layer neural network models have shown to be helpful for high-speed driving when combining them with model predictive path integral control (MPPI) [1], [4], [5] or a simple feedforward-feedback control [6]. However, in order to run MPPI online in a receding horizon manner in a fast control loop, the optimization requires a highly parallel sampling scheme and heavily relies on graphics processing units (GPUs).

The second trend pertains to end-to-end vehicle control using neural networks, where a trained neural network policy can map high-dimensional raw observations directly to control commands. Hence, it has the advantage of forgoing the need for explicit state estimation and solving optimizations online, rendering fast and adaptive control performance. Combined with advanced dynamic model identification, neural network controllers have shown numerous achievements in many real-world robot applications [7]–[10].

In this work, we tackle the autonomous car racing problem in Gran Turismo Sport (GTS), which is a world-leading car simulator known for its realistic modeling of various race cars and tracks. GTS is a unique platform for studying autonomous racing systems, as it allows simulating a large number of realistic environments and benchmarking comparisons between autonomous systems and experienced human drivers. Such comparisons can offer invaluable insights into vehicle control during extreme manoeuvres.

Our approach makes use of model-free deep reinforcement learning and a course-progress proxy reward to train a multilayer perceptron policy for the vehicle control. The key is formulating the minimum-time racing problem properly into maximizing the course progress and choosing meaningful low-dimensional state representations for efficient policy training. As a result, we demonstrate the first super-human control performance on the task of autonomous car racing. The strength of our controller is two-fold: 1) it does not rely on high-level trajectory planning and following (which human drivers tend to do) 2) it generates trajectories qualitatively similar to those chosen by the best human drivers while outperforming the best known human lap times. Our findings suggest that reinforcement learning and neural network controllers merit further investigation for controlling autonomous vehicles at their limits of handling.

II. RELATED WORK

Prior work in the domain of autonomous racing can be grouped into three groups: (i) classical approaches relying on trajectory planning and following, (ii) supervised learning approaches, and (iii) reinforcement learning approaches.

Classical Approaches Classical approaches to autonomous car racing approach the problem by separating it in a chain of submodules consisting of perception, trajectory planning, and control. In particular, model predictive control (MPC) [2],

[11]–[15] is a promising approach for controlling the vehicle at high speed. In [16], an MPC controller is combined with learned system dynamics based on Gaussian Processes for the task of autonomous car racing. While being able to capture the complex dynamics of the car, the resulting controller requires trading-off dynamical fidelity against computation time. Similarly, model predictive path integral control (MPPI) [1], [4], [5] is a more flexible approach that can be combined with complex cost formulations and neural network vehicle models. Both MPC and MPPI have shown impressive results in controlling physical vehicles at high speed in the real-world. Despite their successes, both approaches have limitations, such as the lack of flexibility in the cost function design or the requirement of highly parallel computations. Moreover, hierarchical approaches are also susceptible to failures of each submodule, where the operational states of each module are limited by a series of approximations and linearizations.

Imitation Learning Instead of planning trajectories and tracking them with a controller, imitation-based approaches directly learn a mapping from observation to control action in a supervised fashion. Learning such mapping requires labelled data, which is typically provided by human expert demonstrations or a classical planning and control pipeline. For example, ALVINN (Autonomous Land Vehicle in a Neural Network) [17] is one of the first autonomous driving systems that uses a neural network to follow a road. Similarly, a convolutional neural network (CNN) controller was trained by [18] for lane and road following. Pan et al. [19] use imitation learning for agile, off-road autonomous driving. Distilling human or algorithmic expert data in a learned policy is a promising approach to overcome the strict real-time constraint of classical approaches, but its performance is, by design, upper-bounded by the quality of the training data.

Reinforcement Learning Model-free reinforcement learning optimizes parametrized policies directly based on sampled trajectories, and hence, does not suffer from issues that hamper the aforementioned approaches, such as the necessity of solving nonlinear optimizations online or the dependence on labelled training data. For example, a number of studies [20]–[24] have demonstrated the success of using model-free deep RL for end-to-end driving and racing. Recently, a high-speed autonomous drifting system [24] was developed in simulation using the soft actor-critic (SAC) algorithm [25]. Here, off-policy training plays an important role in [21], [22], [24], in which the high sample complexity previously limiting the widespread adoption of deep RL methods in high-dimensional domains, was substantially reduced by combining offline training with an experience replay buffer. Despite the successful application of deep RL algorithms in real as well as simulated autonomous driving, there is, to the best of our knowledge, no work matching or exceeding the performance of human expert drivers in terms of speed.

III. METHODOLOGY

Our main goal is to build a neural network controller that is capable of autonomously navigating a race car without prior knowledge about the car's dynamics while minimizing

真实世界驾驶生成的数据。

使用现实世界数据改进车辆模型以及处理更复杂的成本公式和一般非线性动力学的要求促使研究人员设计基于学习的系统。第一个与动态模型识别有关。特别是，多层神经网络模型与模型预测路径积分控制（MPPI）[1]、[4]、[5]或简单的前馈-反馈控制[6]相结合时，已被证明有助于高速驾驶。然而，为了在快速控制循环中以水平后退方式在线运行 MPPI，优化需要高度并行的采样方案，并且严重依赖图形处理单元（GPU）。

第二个趋势涉及使用神经网络的端到端车辆控制，其中经过训练的神经网络策略可以将高维原始观察结果直接映射到控制命令。因此，它的优点是无需显式状态估计和在线求解优化，从而呈现快速和自适应的控制性能。与先进的动态模型识别相结合，神经网络控制器在许多现实世界的机器人应用中取得了巨大的成就[7]-[10]。

在这项工作中，我们解决了 Gran Turismo Sport (GTS) 中的自动驾驶赛车问题，GTS 是世界领先的汽车模拟器，以其对各种赛车和赛道的真实建模而闻名。GTS 是研究自动驾驶赛车系统的独特平台，因为它可以模拟大量现实环境，并在自动驾驶系统和经验丰富的人类驾驶员之间进行基准比较。这种比较可以为极端操纵过程中的车辆控制提供宝贵的见解。

我们的方法利用无模型深度强化学习和课程进度代理奖励来训练用于车辆控制的多层感知器策略。关键是将最短时间竞赛问题正确地表述为最大化课程进度并选择有意义的低维状态表示以进行有效的政策培训。结果，我们在自动驾驶赛车任务中首次展示了超人类的控制性能。我们的控制器的优势有两个：1) 它不依赖于高级轨迹规划和跟踪（人类驾驶员往往会这样做）2) 它生成的轨迹在质量上与最优秀的人类驾驶员选择的轨迹相似，同时超越了最知名的人类单圈时间。我们的研究表明，强化学习和神经网络控制器值得进一步研究，以控制自动驾驶车辆的操控极限。

二.相关工作

自动驾驶赛车领域的先前工作可以分为三组：(i) 依赖于轨迹规划和跟踪的经典方法，(ii) 监督学习方法，以及 (iii) 强化学习方法。

经典方法 自动赛车的经典方法通过将问题分解为由感知、轨迹规划和控制组成的一系列子模块来解决该问题。特别是模型预测控制（MPC）[2]，

[11]-[15]是一种有前途的高速控制车辆的方法。在[16]中，MPC控制器与基于高斯过程的学习系统动力学相结合，用于自动赛车任务。虽然能够捕获汽车的复杂动态，但最终的控制器需要在动态保真度和计算时间之间进行权衡。同样，模型预测路径积分控制（MPPI）[1]、[4]、[5]是一种更灵活的方法，可以与复杂的成本公式和神经网络车辆模型相结合。MPC 和 MPPI 在现实世界中高速控制物理车辆方面都表现出了令人印象深刻的结果。尽管取得了成功，但这两种方法都有局限性，例如成本函数设计缺乏灵活性或需要高度并行计算。此外，分层方法还容易受到每个子模块故障的影响，其中每个模块的操作状态受到一系列近似和线性化的限制。

模仿学习基于模仿的方法不是规划轨迹并用控制器跟踪它们，而是直接学习从观察到以监督方式控制动作的映射。学习此类映射需要标记数据，这些数据通常由人类专家演示或经典规划和控制管道提供。例如，ALVINN（神经网络中的自主陆地车辆）[17]是最早使用神经网络沿着道路行驶的自动驾驶系统之一。类似地，[18]训练了卷积神经网络（CNN）控制器用于车道和道路跟踪。潘等人。[19]使用模仿学习来实现敏捷的越野自动驾驶。在学习策略中提取人类或算法专家数据是克服经典方法严格实时约束的一种有前途的方法，但从设计上讲，其性能受到训练数据质量的上限。

强化学习 无模型强化学习直接根据采样轨迹优化参数化策略，因此不会遇到阻碍上述方法的问题，例如在线解决非线性优化的必要性或对标记训练数据的依赖。例如，许多研究 [20]-[24] 已经证明了使用无模型深度强化学习进行端到端驾驶和赛车的成功。最近，使用软演员批评家（SAC）算法[25]在模拟中开发了高速自主漂移系统[24]。在这里，离线训练在[21]、[22]、[24]中发挥着重要作用，其中之前限制深度强化学习方法在高维领域广泛采用的高样本复杂性，通过将离线训练与经验重播缓冲区相结合而大大降低。尽管深度强化学习算法在真实和模拟自动驾驶中取得了成功应用，但据我们所知，在速度方面还没有任何工作能够与人类专家驾驶员的表现相匹配或超过。

三. 方法论

我们的主要目标是构建一个神经网络控制器，能够在不事先了解汽车动力学的情况下自动导航赛车，同时最大限度地减少

the traveling time on a given track in the GTS environment. To achieve this goal, we first define a reward function that formulates the racing problem and a neural network policy that maps input states to actions. We then optimize the policy parameters by maximizing the reward function using the SAC [25] algorithm. An overview of our system is shown in Figure 2.

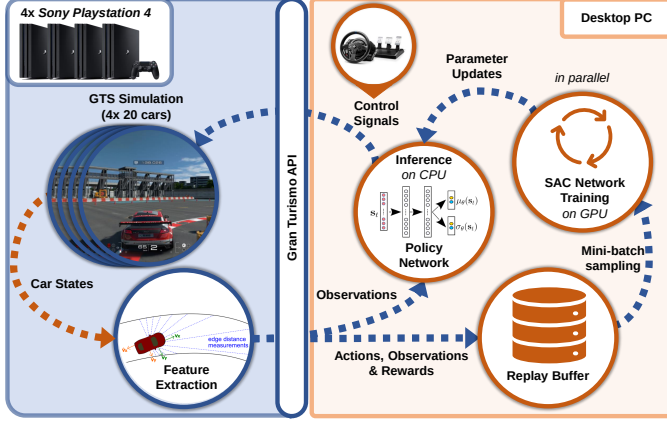


Fig. 2. System overview: we train a policy network to directly map observations, including a set of range finder measurements and the car’s velocity and acceleration, to control commands consisting of the car’s steering angle as well as the level of throttle and brake. We use a distributed sampling scheme to collect samples from 4 *PlayStation 4* simulating 20 cars each, and then, store the sampled trajectory in a fixed size first in, first out (FIFO) replay buffer. In parallel, we optimize the policy parameters using a soft actor-critic algorithm and data uniformly sampled from the replay buffer.

A. Minimum-Time Problem and Reward Function

We aim to find a policy that minimizes the total lap time for a given car and track. The lap time itself is a very sparse reward signal. Changes in the signal are therefore hard to attribute to specific actions of an agent. We design a proxy reward based on the current course progress, which can be evaluated in arbitrary time intervals. To reduce the variance of the reward signal, we use an exponential discount for future rewards, with discount factor γ . Maximizing the course progress closely approximates minimizing the lap time when choosing a high enough discount factor. The new proxy reward allows trading-off an easily attributable but biased reward and a reward closer to the overall lap time goal by adjusting γ , and thus changing the weight of future rewards based on their temporal distance to an action and state. The construction of the progress reward can be seen in the top of Figure 3.

The use of an exponentially discounted future reward leads to a bias towards short-term rewards. In the GTS setting this bias reduces the incentive for an RL agent to brake, e.g. to prevent crashes. To counteract this short-term bias, we introduce a second reward term that penalizes wall contact relative to the car’s kinetic energy, leading to the final reward function of

$$r_t = r_t^{\text{prog}} - \begin{cases} c_w \|\mathbf{v}_t\|^2 & \text{if in contact with wall,} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where $c_w \geq 0$ is a hyperparameter controlling the trade off between wall contact penalty and progress-based reward. Here

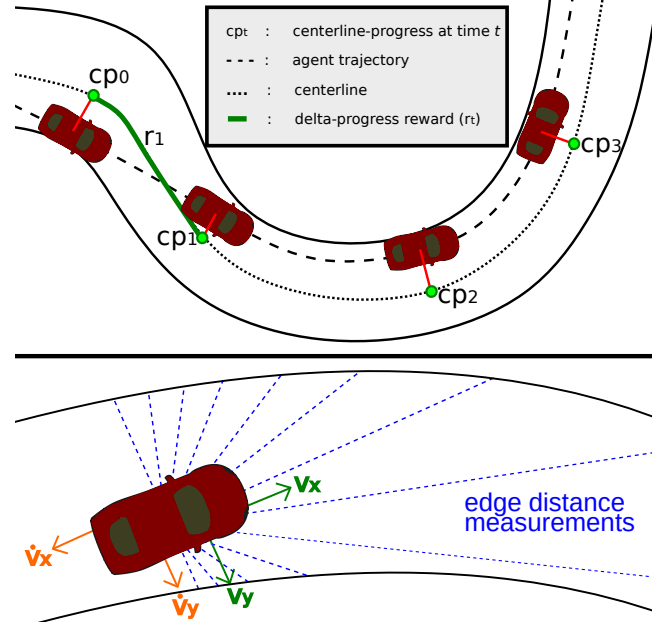


Fig. 3. **Top:** The course progress cp_t at time t is constructed by projecting the car’s position on the track’s center line. We then define the progress-based reward r^{prog} as the increment in centerline-progress $r_t^{\text{prog}} = cp_t - cp_{t-1}$. **Bottom:** A subset of the observations fed to the policy and value networks.

$\mathbf{v}_t = [v_x, v_y, v_z]$ is a velocity vector that represents the current linear velocity of the vehicle. A similar approach to incentivize collision avoidance was introduced in [26]. The introduction of the kinetic energy is justified through the energy-dependent loss in acceleration that takes place when hitting a wall. Without this additional wall contact penalty, we found the learned policies did not brake and simply grinded along the track’s walls in sharp curves. When using fixed valued wall contact penalties, we found the agent either did not react to the penalty or ended up in a strategy of full braking and standing still to not risk any wall contact, depending on the strength of the penalty.

B. Policy Network

We represent the driving policy with a deep neural network. We make use of the SAC network architecture proposed in the original paper [25]. We use a policy network, two Q-function networks, and a state-value function network, each with 2 hidden layers with 256 ReLU nodes, resulting in a total of 599,566 trainable parameters. In the following, the features fed to the network and its prediction are explained in detail.

Input features We base the input feature selection on the results of a pre-study on GTS conducted with behavioral cloning, aiming to learn human-like driving based on a regression of experts’ actions on visited states. We iterated over combinations of subsets of features provided by GTS as well as additionally constructed features, which lead to the following feature selection:

- 1) The linear velocity $\mathbf{v}_t \in \mathbb{R}^3$ and the linear acceleration $\dot{\mathbf{v}}_t \in \mathbb{R}^3$.
- 2) The Euler angle $\theta_t \in (-\pi, \pi]$ between the 2D vector that defines the agent’s rotation in the horizontal plane and the unit tangent vector that is tangent to the centerline at

GTS 环境中给定轨道上的行驶时间。为了实现这个目标，我们首先定义一个奖励函数来制定赛车问题，以及一个将输入状态映射到动作的神经网络策略。然后，我们使用 SAC [25] 算法最大化奖励函数来优化策略参数。我们的系统概述如图 2 所示。

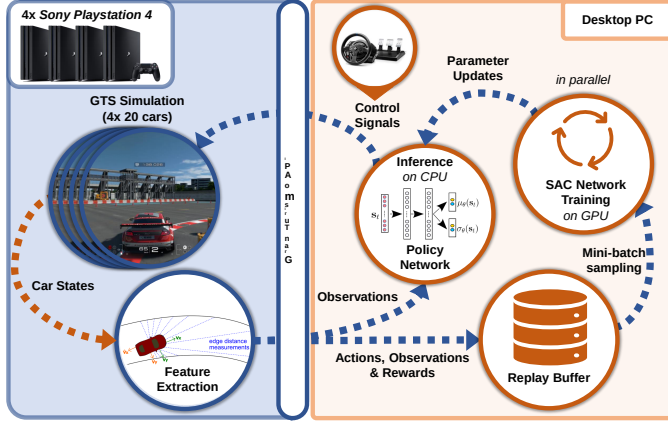


图 2. 系统概述：我们训练一个策略网络来直接映射观测结果，包括一组测距仪测量结果以及汽车的速度和加速度，以控制由汽车转向角以及油门和制动级别组成的命令。我们使用分布式采样方案从 4 个 PlayStation 4 中收集样本，每个 PlayStation 4 模拟 20 辆汽车，然后将采样的轨迹存储在固定大小的先进先出 (FIFO) 重播缓冲区中。同时，我们使用软演员批评算法和从重播缓冲区均匀采样的数据来优化策略参数。

A. Minimum-Time Problem and Reward Function

我们的目标是找到一种策略，最大限度地减少给定赛车和赛道的总单圈时间。单圈时间本身是一个非常稀疏的奖励信号。因此，信号的变化很难归因于代理的特定行为。我们根据当前课程进度设计一个代理奖励，可以在任意时间间隔内进行评估。为了减少奖励信号的方差，我们对未来奖励使用指数折扣，折扣因子为 γ 。当选择足够高的折扣系数时，最大化路线进度非常接近最小化单圈时间。新的代理奖励允许通过调整 γ 来权衡易于归因但有偏差的奖励和更接近总体单圈时间目标的奖励，从而根据与动作和状态的时间距离改变未来奖励的权重。进度奖励的构建如图 3 顶部所示。

使用指数折扣的未来奖励会导致对短期奖励的偏见。在 GTS 设置中，这种偏差会降低 RL 智能体制动的动机，例如以防止崩溃。为了抵消这种短期偏差，我们引入了第二个奖励项，它相对于汽车的动能来惩罚墙壁接触，从而导致最终的奖励函数为

$$r_t = r_t^{\text{prog}} - \begin{cases} c_w \|\mathbf{v}_t\|^2 & \text{if in contact with wall,} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

其中 $c_w \geq 0$ 是一个超参数，控制墙壁接触惩罚和基于进度的奖励之间的权衡。这里

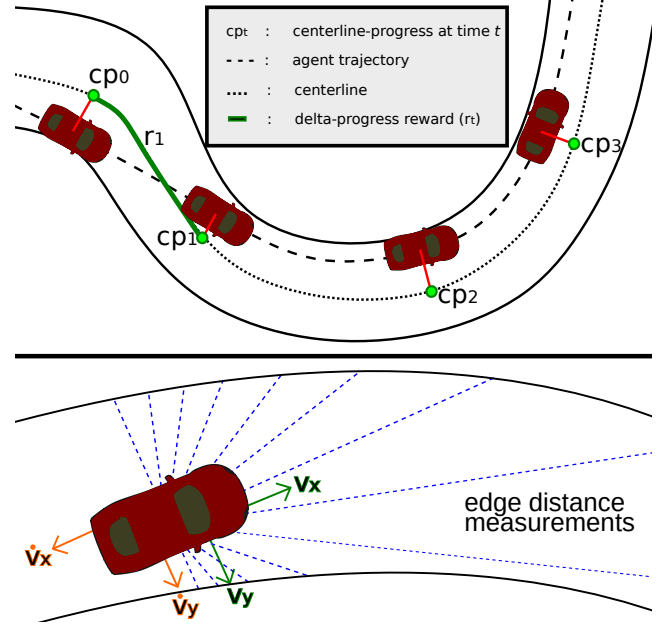


图 3. 顶部：时间 t 的路线进度 cp_t 是通过将汽车的位置投影到赛道中心线上来构建的。然后，我们将基于进度的奖励 r^{prog} 定义为中心线进度 $r_t^{\text{prog}} = cp_t - cp_{t-1}$ 的增量。底部：馈送到政策和价值网络的观察子集。

$\mathbf{v}_t = [v_x, v_y, v_z]$ 是代表车辆当前线速度的速度矢量。[26] 中引入了类似的激励避免碰撞的方法。动能的引入是通过碰撞墙壁时发生的与能量相关的加速度损失来证明的。如果没有这种额外的墙壁接触惩罚，我们发现学习的策略不会制动，只是沿着赛道的墙壁以锐曲线磨擦。当使用固定值的墙壁接触惩罚时，我们发现智能体要么没有对惩罚做出反应，要么最终采取完全制动和静止不动的策略，以不冒任何墙壁接触的风险，具体取决于惩罚的强度。

B. Policy Network

我们用深度神经网络来表示驾驶策略。我们利用原始论文 [25] 中提出的 SAC 网络架构。我们使用一个策略网络、两个 Q 函数网络和一个状态值函数网络，每个网络都有 2 个隐藏层和 256 个 ReLU 节点，总共产生 599,566 个可训练参数。下面详细解释输入网络的特征及其预测。

输入特征 我们根据通过行为克隆对 GTS 进行预研究的结果来选择输入特征，旨在根据专家对访问状态的行为进行回归来学习类人驾驶。我们迭代了 GTS 提供的特征子集以及额外构造的特征的组合，这导致了以下特征选择：

- 1) 线速度 $\mathbf{v}_t \in \mathbb{R}^3$ 和线加速度 $\dot{\mathbf{v}}_t \in \mathbb{R}^3$ 。2) 定义智能体在水平面内旋转的 2D 向量与与中心线相切的单位切向量之间的欧拉角 $\theta_t \in (-\pi, \pi]$

the projection point. This angle is the only direct way for the policy network to detect if a car is facing the wrong direction. 3) Distance measurements $\mathbf{d}_t \in \mathbb{R}^M$ of M rangefinders with a maximum range of 100m that measure the distance from the vehicle's center point to the M edge points of its surrounding objects, such as the edge of the race track. The rangefinders are equally distributed in the front 180° of the car's view. 4) The previous steering command δ_{t-1} . 5) A binary flag with $w_t = 1$ indicating wall contact and 6) N sampled curvature measurement of the course centerline in the near future $\mathbf{c}_t \in \mathbb{R}^N$. The curvature is represented through an interpolation of the inverse radii of circles fitted through centerline points provided by GTS.

We therefore represent the observation at a given time step t as a vector denoted as $\mathbf{s}_t = [\mathbf{v}_t, \dot{\mathbf{v}}_t, \theta_t, \mathbf{d}_t, \delta_{t-1}, w_t, \mathbf{c}_t]$. A subset of the features is illustrated in the bottom of Figure 3. To allow for a fair comparison with human drivers, we only use features that humans can either directly perceive or deduce from the GTS simulation.

Network prediction The output of the policy network $\mathbf{a}_t = [\delta_t, \omega_t]$ directly encodes the steering angle $\delta_t \in [-\pi/6, \pi/6]$ rad and a combined throttle-brake signal $\omega_t \in [-1, 1]$, where $\omega_t = 1$ denotes full throttle and $\omega_t = -1$ full braking. The combination of throttle and brake in a single signal is motivated by an analysis of human recordings, which revealed that fast strategies do not involve any simultaneous use of throttle and brake.

IV. EXPERIMENTS

We evaluate our approach in three race settings, featuring different cars and tracks of varying difficulty. We compare our approach to the built-in AI as well as a set of over 50,000 human drivers. To ensure a fair comparison to the human drivers, we constrain the policy to produce only actions that would also be feasible with a physical steering wheel. In the following, each race setting as well as the constraints imposed for the comparison with human drivers are explained.

Race settings We train separate agents for three experimental conditions for which human data from past online time trial competitions is available. The data, provided by GTS manufacturer *Polyphony Digital Inc.*, includes the personal best lap time and trajectory of every competitor, ranging from absolute beginners to World Cup contenders. The competitions were restricted to fixed settings such as car, course, tires, and racing assistance settings. This allows for equal conditions when comparing human results with our approach. Figure 4 shows the used cars and tracks. Setting A and B use a track layout featuring a typical combination of curves and straight segments. The two settings differ in the cars used, with the ‘‘Audi TT Cup ’16’’ of setting A having a higher maximum speed, more tire grip, and a faster acceleration than the ‘‘Mazda Demio XD Turing ’15’’ of setting B. Setting C features the same car as setting A but a more challenging track layout which features a larger range of speeds due to the combination of long straight segments and very tight turns.

Leveling the playing field Whereas human players are limited by the physical inertia of their game controllers, our

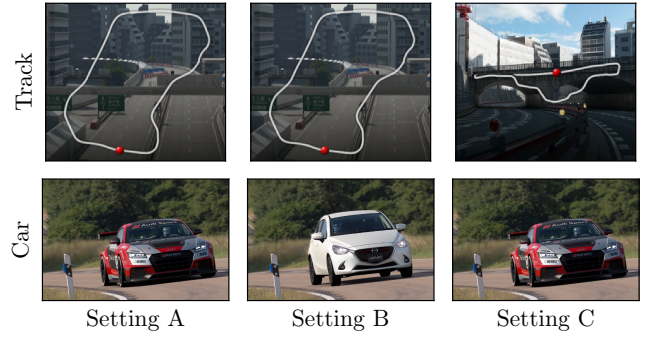


Fig. 4. The tracks and cars used as reference settings to compare our approach to human drivers.

agent can, in principle, make arbitrarily large action changes. To allow for a fair comparison between our agent and human players, we estimated the maximal action change that a human can achieve within one frame with a *Thrustmaster T300* steering wheel and pedals. Based on that, we restricted the maximum change between frames during evaluation to 0.03 rad for the steering angle and 80% for each the throttle and brake range¹. We however found this restriction to not significantly influence the resulting lap times.

Robustness To detect potential shortcoming of our approach for a more general driving task, such as driving on multiple tracks or driving in the real world, we evaluate our agent's robustness by testing its ability to generalize to a range of modified environments. We deploy the fastest agent trained in setting A on the following problem settings without any retraining: 1) transferring the policy to a different car (setting B), 2) transferring the policy to a different track (setting C), 3) changing the tire friction, 4) adding uniform noise to the agent's observations, and 5) delaying the agent's inference.

V. RESULTS

In this section we evaluate our approach on the three reference settings introduced in Section IV. We compare the lap times obtained by our approach with the ones achieved by AI built into GTS and the fastest human drivers. Additionally, we analyze the resulting driving behaviour and compare it with driving strategies of professional human drivers. Due to the dynamic nature of our experiments, we encourage the reader to watch the supplementary videos.

A. Lap Time Comparison

Our approach outperforms the best human lap time in all three reference settings, overcoming the limitations of the currently built-in AI, which itself is outperformed by a majority of players. Table I shows the fastest achieved lap times for the three race settings.

On the first track our approach undercuts the fastest human lap times by 0.15 and 0.04 seconds for settings A and B respectively. We believe that the difference in margins to the best human drivers results from the speed differences between

¹A similar fairness restriction was put on the agent proposed in [27], by limiting the number of actions an agent could take per minute.

投影点。这个角度是策略网络检测汽车是否面向错误方向的唯一直接方式。3) M 测距仪的距离测量 $\mathbf{d}_t \in \mathbb{R}^M$ ，最大范围为100 m，测量从车辆中心点到周围物体的 M 边缘点（例如赛道边缘）的距离。测距仪均匀分布在汽车视野前方 180° 范围内。4) 前一个转向命令 δ_{t-1} 。5) 一个二元标志，其中 $w_t = 1$ 指示壁接触，并且 6) N 在不久的将来 $\mathbf{c}_t \in \mathbb{R}^N$ 采样了球场中心线的曲率测量值。曲率通过 GTS 提供的中心线点拟合的圆的倒数半径插值来表示。

因此，我们将给定时间步 t 的观察表示为表示为 $\mathbf{s}_t = [\mathbf{v}_t, \dot{\mathbf{v}}_t, \theta_t, \mathbf{d}_t, \delta_{t-1}, w_t, \mathbf{c}_t]$ 的向量。图 3 底部显示了这些特征的子集。为了与人类驾驶员进行公平比较，我们仅使用人类可以直接感知或从 GTS 模拟中推断出的特征。

网络预测策略网络 $\mathbf{a}_t = [\delta_t, \omega_t]$ 的输出直接编码转向角 $\delta_t \in [-\pi/6, \pi/6]$ rad和组合油门制动信号 $\omega_t \in [-1, 1]$ ，其中 $\omega_t = 1$ 表示全油门， $\omega_t = -1$ 表示全制动。将油门和刹车组合在一个信号中的动机是对人类记录的分析，该分析表明快速策略不涉及油门和刹车的任何同时使用。

四. 实验

我们在三种比赛环境中评估我们的方法，包括不同的赛车和不同难度的赛道。我们将我们的方法与内置人工智能以及超过 50,000 名人类驾驶员进行比较。为了确保与人类驾驶员进行公平比较，我们限制策略仅产生使用物理方向盘也可行的操作。下面，解释每场比赛的设置以及与人类车手进行比较所施加的限制。

比赛设置 我们针对三种实验条件训练单独的智能体，在这些实验条件下，可以获得过去在线计时赛的人类数据。这些数据由 GTS 制造商 *Polyphony Digital Inc.* 提供，包括每位参赛者的个人最佳单圈时间和轨迹，从绝对的初学者到世界杯竞争者。比赛仅限于固定设置，如赛车、赛道、轮胎和赛车辅助设置。在将人类结果与我们的方法进行比较时，这允许相同的条件。图 4 显示了二手车和轨道。设置A和B使用具有典型的曲线和直线段组合的轨道布局。这两种设置因所使用的车辆而异，设置为A的“Audi TT Cup ’16”比设置为B的“Mazda Demio XD Turing ’15”具有更高的最大速度、更大的轮胎抓地力和更快的加速度。设置C的车辆与设置A的车辆相同，但赛道布局更具挑战性，由于长直线段和非常急转弯的结合，速度范围更大。

公平竞争环境虽然人类玩家受到游戏控制器物理惯性的限制，但我们的

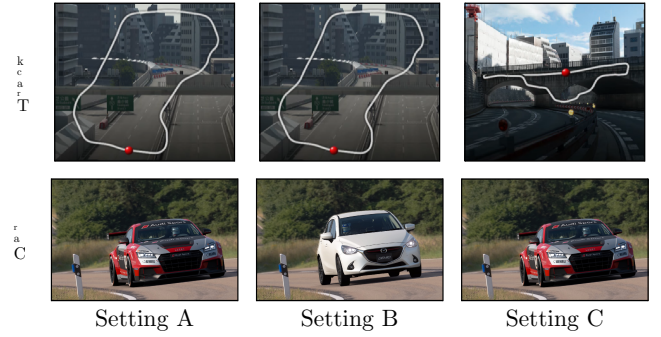


图 4. 赛道和汽车用作参考设置，以将我们的方法与人类驾驶员进行比较。

原则上，智能体可以做出任意大的动作改变。为了在我们的代理和人类玩家之间进行公平的比较，我们估计了人类使用 *Thrustmaster T300* 方向盘和踏板在一帧内可以实现的最大动作变化。基于此，我们将评估过程中帧之间的最大变化限制为转向角 0.03 rad ，油门和制动范围¹的最大变化限制为 80% 。然而，我们发现这一限制并没有显著影响最终的单圈时间。

鲁棒性为了检测我们的方法在更一般的驾驶任务中的潜在缺点，例如在多条轨道上驾驶或在现实世界中驾驶，我们通过测试其泛化到一系列修改环境的能力来评估我们的代理的鲁棒性。我们部署了在以下问题设置上设置A训练的最快代理，无需任何重新训练：1) 将策略转移到不同的汽车（设置B），2) 将策略转移到不同的赛道（设置C），3) 改变轮胎摩擦力，4) 向代理的观察添加均匀的噪声，5) 延迟代理的推理。

五. 结果

在本节中，我们根据第四节中介绍的三个参考设置评估我们的方法。我们将通过我们的方法获得的单圈时间与 GTS 内置的人工智能和最快的人类驾驶员所获得的单圈时间进行比较。此外，我们还分析了由此产生的驾驶行为，并将其与专业人类驾驶员的驾驶策略进行比较。由于我们实验的动态性质，我们鼓励读者观看补充视频。

A. Lap Time Comparison

我们的方法在所有三种参考设置中都优于人类的最佳单圈时间，克服了当前内置人工智能的局限性，而大多数玩家的表现都优于当前内置人工智能。表一显示了三种比赛设置下最快的单圈时间。

在第一条赛道上，我们的方法在设置A和B时分别将人类最快单圈时间缩短了 0.15 s 和 0.04 s 。我们相信，与最优秀的人类驾驶员之间的差距是由于之间的速度差异造成的。

¹A similar fairness restriction was put on the agent proposed in [27], by limiting the number of actions an agent could take per minute.

TABLE I
TIME TRIAL COMPARISONS BETWEEN OUR APPROACH, HUMAN ONLINE COMPETITORS, AND THE BUILT-IN GTS AI FOR THE 3 RACE SETTINGS.

Driver	Metric	Setting A	Setting B	Setting C
Ours	Lap time [min]	01:15.913	01:39.408	02:06.701
Human players	Fastest lap [min]	01:16.062	01:39.445	02:07.319
	Median lap [min]	01:22.300	01:47.259	02:13.980
	# participants	52,303	28,083	52,335
Built-in GTS AI	Lap time [min]	01:26.899	01:52.075	02:14.252
	Slower than x% of humans drivers	82.6%	80.4%	53.9%

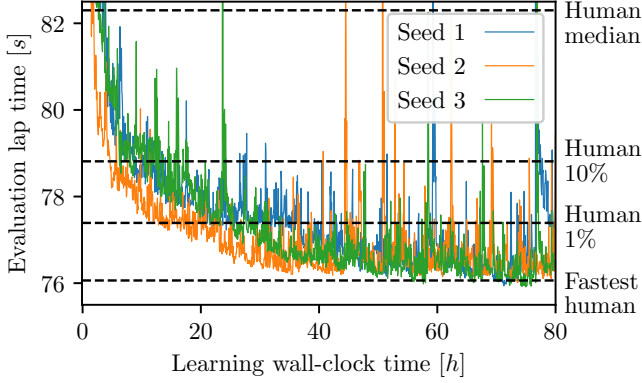


Fig. 5. Training progress for our approach learning on setting A. We evaluate the policy every 2 epochs of learning by letting the agent drive 2 consecutive laps. We take the lap time at the second lap as performance metric such that the fixed starting speed applied in the first lap does not distort the result.

the two used cars, with the faster “Audi TT Cup” in setting A requiring a more agile strategy than the relatively slow “Mazda Demio” in setting B. While human players possibly struggle with the fast-paced setting, our approach does not suffer under the increased demands. Due to the similarities of the trajectories of setting A and B, in the following part we will only analyze the trajectories of setting A. Over 10 consecutive laps in setting A our agent’s lap time shows a standard deviation of 15 ms. A top 1% driver we invited for our study showed a standard deviation of 480 ms. In setting C our approach undercuts the best human time by 0.62 seconds. As in the other two settings, the increased margin can be explained by the more challenging track layout and faster car that render setting C the most difficult combination in our experiments.

Figure 5 shows the learning progress in setting A for three differently initialized neural network policies. With each random initialization our approach learns policies that achieve lap times faster than the fastest human reference lap after 56 to 73 hours of training, which on average corresponds to 2,151 training epochs and a total of 946,453 km driven. The 3 seeds result in similar learning progresses that mainly differ in the time and order in which they learn to master certain curves. Note that warm-starting the training process from a policy that is trained via imitation learning from human data does not improve final performance. Training the policy from scratch outperforms the behaviour-cloning policy after less than one hour of training.

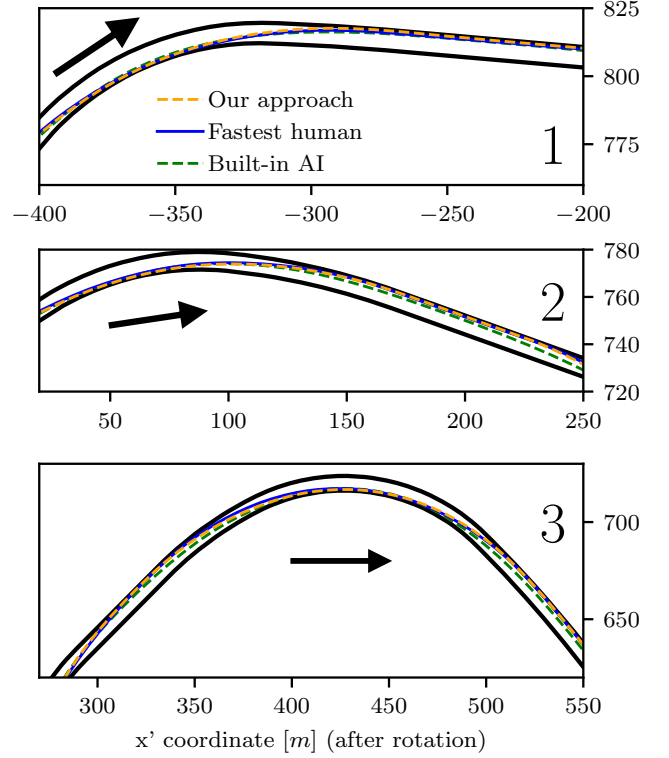


Fig. 6. Top-down view of driven paths for setting A. While the paths of our approach and the human driver are similar, the built-in GTS AI drives with a safety margin to the track’s walls, leading to tighter radii and a reduced speed.

B. Learned Driving Behavior

We analyze the models’ driving behavior at the epoch with the fastest evaluation lap time.

Out-in-out trajectory Our approach learned to make use of the whole width of the track to maximize its trajectories’ curve radii, driving a so-called out-in-out trajectory. This allows the agent to drive higher speeds before losing traction. Figure 6 shows that learned behavior in a curve driving comparison between our approach, the fastest human, and the built-in AI for 3 curves in setting A. Our approach learned to drive curves similar to those of the fastest human, without having access to any human demonstration or using an explicit path planning mechanism. The top of Figure 7 shows the learned out-in-out trajectory for the sharpest curve of setting A.

Anticipating curves Furthermore, our approach learned to detect curves early enough and assess their sharpness to decelerate to speeds that allow completing the curves without overshooting into the track’s walls, while at the same time not being overly cautious. This can be seen in the sharpest curve of setting A in the top of Figure 7 and in a more extreme case in the hairpin of setting C in the bottom of the Figure. In the hairpin, the agent starts decelerating ~ 100 meters before the start of the curve.

Overall speed The top of Figure 8 shows a speed comparison between our approach, the fastest human lap, and the built-in AI in setting A. Our approach learned a control policy that closely matches and sometimes even improves on the speed

表 I 我们的方法、人类在线竞争对手和内置 GTS AI 在 3 种比赛设置下的计时赛比较。

Driver	Metric	Setting A	Setting B	Setting C
Ours	Lap time [min]	01:15.913	01:39.408	02:06.701
Human players	Fastest lap [min]	01:16.062	01:39.445	02:07.319
	Median lap [min]	01:22.300	01:47.259	02:13.980
	# participants	52,303	28,083	52,335
Built-in GTS AI	Lap time [min]	01:26.899	01:52.075	02:14.252
	Slower than x% of humans drivers	82.6%	80.4%	53.9%

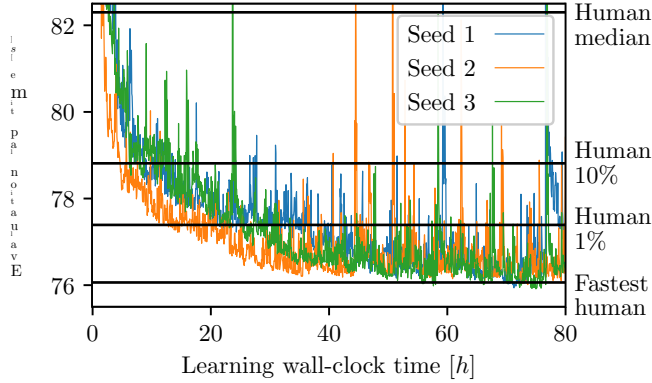


图 5. 我们在设置 A 上学习的方法的训练进度。我们通过让智能体连续行驶 2 圈来每 2 个学习周期评估策略。我们将第二圈的单圈时间作为性能指标，以便第一圈应用的固定起始速度不会扭曲结果。

两辆二手车，在设置 A 中速度更快的“奥迪 TT Cup”比在设置 B 中相对较慢的“马自达 Demio”需要更敏捷的策略。虽然人类玩家可能会在快节奏的环境中挣扎，但我们的方法不会因需求的增加而受到影响。由于设置 A 和 B 的轨迹相似，在下面的部分中我们将只分析设置 A 的轨迹。在设置 A 的连续 10 圈中，我们的代理的单圈时间显示出 15 毫秒的标准偏差。我们邀请参加研究的前 1% 驾驶员的标准偏差为 480 毫秒。在设置 C 时，我们的方法将人类最佳时间缩短了 0.62 秒。与其他两种设置一样，裕量的增加可以通过更具挑战性的赛道布局 and 更快的赛车来解释，这使得设置 C 成为我们实验中最困难的组合。

图 5 显示了为三种不同初始化的神经网络策略设置 A 的学习进度。通过每次随机初始化，我们的方法都会学习在 56 至 73 小时的训练后实现比最快人类参考单圈更快的单圈时间的策略，这平均对应于 2,151 个训练周期和总共 94 6,453 公里的行驶里程。这 3 颗种子的学习进度相似，主要区别在于它们学习掌握某些曲线的时间和顺序。请注意，从通过人类数据的模仿学习训练的策略中热启动训练过程不会提高最终性能。从头开始训练策略在不到一小时的训练后优于行为克隆策略。

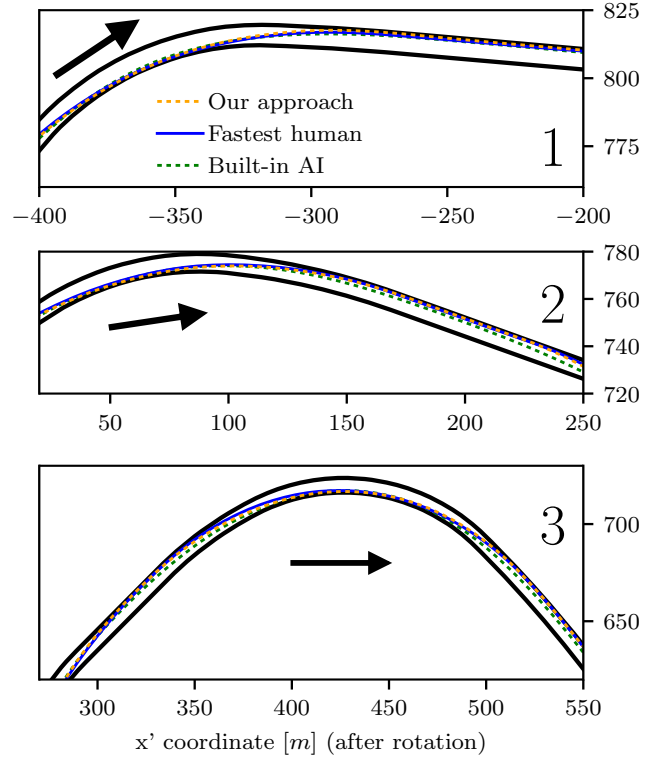


图 6. 用于设置 A 的驱动路径的自上而下视图。虽然我们的接近路径与人类驾驶员相似，但内置的 GTS AI 驾驶时对赛道壁有安全裕度，从而导致半径更小并降低速度。

B. Learned Driving Behavior

我们分析了模型在评估单圈时间最快的时期的驾驶行为。

Out-in-out 轨迹 我们的方法学会了利用轨道的整个宽度来最大化其轨迹的曲线半径，从而驱动所谓的 out-in-out 轨迹。这使得代理可以在失去牵引力之前以更高的速度行驶。图 6 显示了在设置 A 的 3 条曲线中，我们的方法（最快的人类）和内置 AI 之间的曲线驱动学习行为比较。我们的方法学会了驾驶与最快的人类相似的曲线，而无需访问任何人类演示或使用明确的路径规划机制。图 7 的顶部显示了设置 A 的最尖锐曲线的学习出的出入出轨迹。

预测弯道此外，我们的方法学会了尽早检测弯道并评估其锐度，以减速到允许完成弯道而不会过度冲入赛道壁的速度，同时又不会过度谨慎。这可以从图 7 顶部的设置 A 的最尖锐曲线以及图底部的设置 C 的发夹形更极端的情况中看出。在发夹弯处，智能体在弯道开始前 100 米开始减速。

总体速度 图 8 的顶部显示了我们的方法（人类最快圈速）与设置 A 的内置 AI 之间的速度比较。我们的方法学习了一种与速度密切匹配甚至有时甚至提高的控制策略

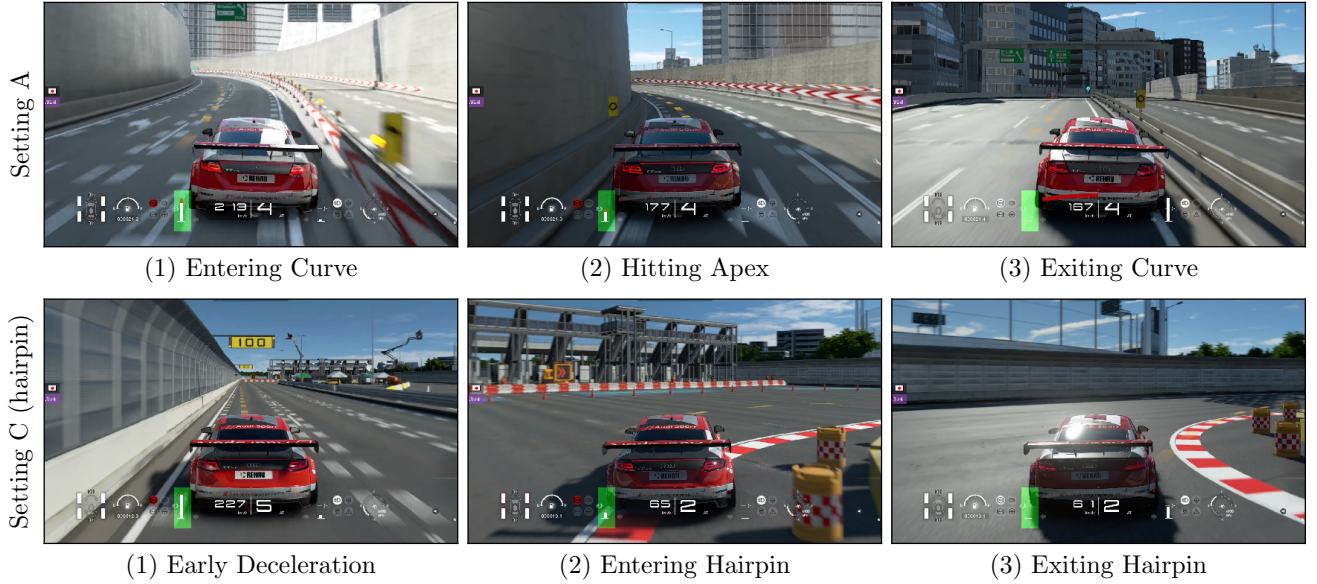


Fig. 7. Out-in-out driving behavior and early curve anticipation learned by our approach for setting A (top) and setting C (bottom). The brake signal is represented by the white bar marked in green. The segments are shown in the supplementary video from 0:24 to 0:30 and 2:50 to 3:04.

as well as the path of the fastest human reference lap. Even though the path and speed of our approach and the fastest human are similar for setting A, our approach undercuts the fastest human lap time by 0.15 seconds. While this might not seem like a significant difference, this margin is similar to those between top contenders in real and simulated racing championships.

the human expert drives a wider trajectory on the incoming straight, our approach compensates on the outgoing straight, leading to a similar curve exit speed and curve completion time. The right side of Figure 9 shows the speed difference between our approach and the fastest human. Our approach achieves similar or higher speeds over most of the track, often driving tighter and faster curves than the human expert, which leads to an overall faster time.

In summary, these results indicate that our approach learns to drive autonomously at high-speed in different race settings, using different cars and driving on different tracks, including a challenging hairpin curve. In all settings our approach achieved lap times faster than those of all human reference drivers. Moreover, we find that our approach learns to drive trajectories that are qualitatively similar to those chosen by the best human players while maintaining slightly higher average speeds in curves by successfully executing later brake points.

C. Robustness

We analyze the robustness of our trained agent by modifying the test environment.

1) Transfer to new car: The agent still follows an out-in-out trajectory and is able to finish the track. In straight segments as well as slight curves, the agent is able to adjust to the change in dynamics and execute a feasible trajectory. In sharp curves however, the agent sometimes overshoots the track. This is most likely caused by the training car being easier to control than the test car, due to higher tire-surface friction caused by both the weight and the racing tires of the training car. For sharper curves, the agent is not able to extrapolate its behavior to the previously unseen dynamics of a car with less tire-surface friction.

2) Transfer to new track: The agent is able to drive wall-contact free in straight segments as well as slight curves. It is however not able to extrapolate its behavior to some of the

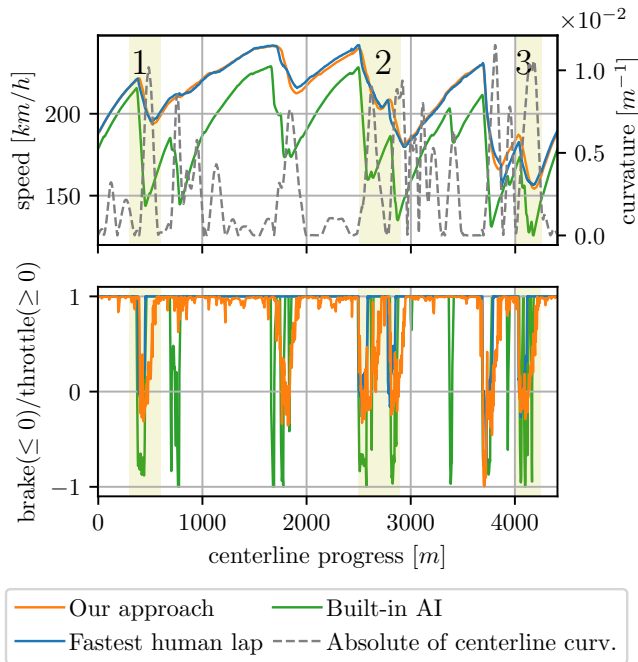


Fig. 8. **Top:** Comparison of speed between the fastest human, our approach, and the built-in GTS AI in setting A. **Bottom:** Comparison of combined throttle/brake signal.

The left side of Figure 9 shows the path and speed driven by our approach and the fastest human in the hairpin curve. While

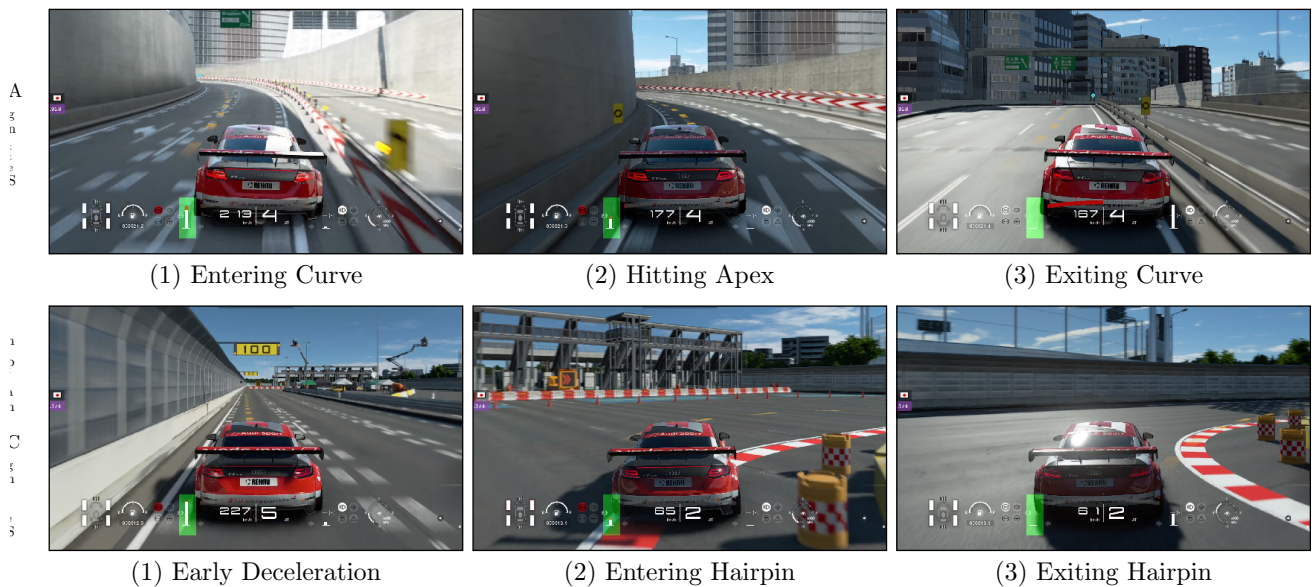


图7。通过我们设置 A (top) 和设置 C (bottom) 的方法学到了由外到外的驾驶行为和早期弯道预测。制动信号为代表 由绿色标记的白色条插入。补充视频中的片段显示为 0:24 至 0:30 和 2:50 至 3:04。

以及人类最快参考圈速的路径。尽管我们的方法和最快的人类在设置 A 时的路径和速度相似，但我们的方法将人类最快的单圈时间缩短了 0.15 秒。虽然这看起来似乎不是一个显着的差异，但这个差距与真实和模拟赛车锦标赛中顶级竞争者之间的差距相似。

人类专家在传入直道上驾驶更宽的轨迹，我们的方法对传出直道进行补偿，从而获得相似的弯道出口速度和弯道完成时间。图 9 的右侧显示了我们的方法与最快的人类之间的速度差异。我们的方法在大部分赛道上实现了相似或更高的速度，通常比人类专家驾驶更紧、更快的弯道，这导致整体时间更快。

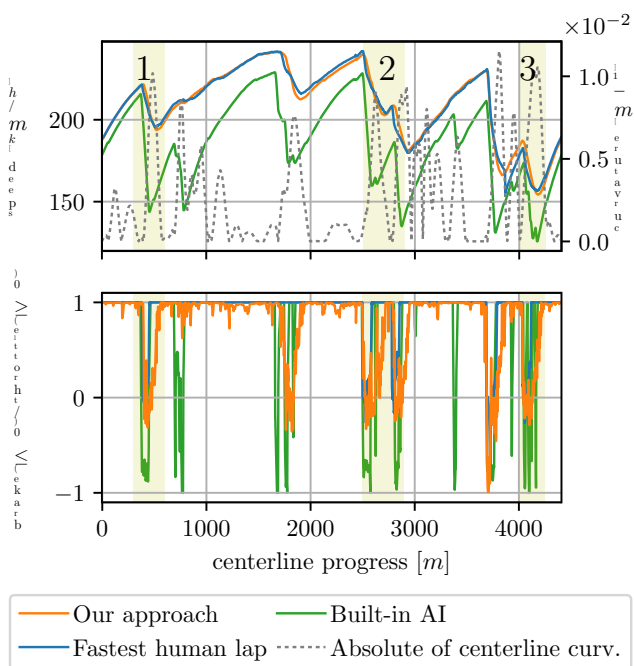


图 8. 顶部：最快的人类、我们的方法和设置 A 中的内置 GTS AI 之间的速度比较。底部：组合油门/制动信号的比较。

图 9 的左侧显示了我们的接近和发夹弯中最快的人类所驱动的路径和速度。尽管

总之，这些结果表明我们的方法学会了在不同的比赛环境中高速自动驾驶，使用不同的汽车并在不同的赛道上行驶，包括具有挑战性的发夹弯道。在所有设置中，我们的方法实现的单圈时间都比所有人类参考驾驶员的圈速更快。此外，我们发现我们的方法学会了驾驶轨迹，其质量与最优秀的人类玩家选择的轨迹相似，同时通过成功执行稍后的制动点来保持弯道中稍高的平均速度。

C. Robustness

我们通过修改测试环境来分析训练有素的代理的稳健性。

1) 转移到新车：智能体仍然遵循从外到外的轨迹，并且能够完成赛道。在直线段和轻微的曲线中，智能体能够适应动态变化并执行可行的轨迹。然而，在急弯中，代理有时会超出赛道。这很可能是由于训练车比测试车更容易控制造成的，因为训练车的重量和赛车轮胎造成了更高的轮胎表面摩擦力。对于更陡的弯道，智能体无法将其行为推断为轮胎表面摩擦力较小的汽车以前未见过的动态。

2) 转移到新轨道：代理能够在直线段以及轻微的弯道中自由行驶。然而，它无法将其行为推断为某些

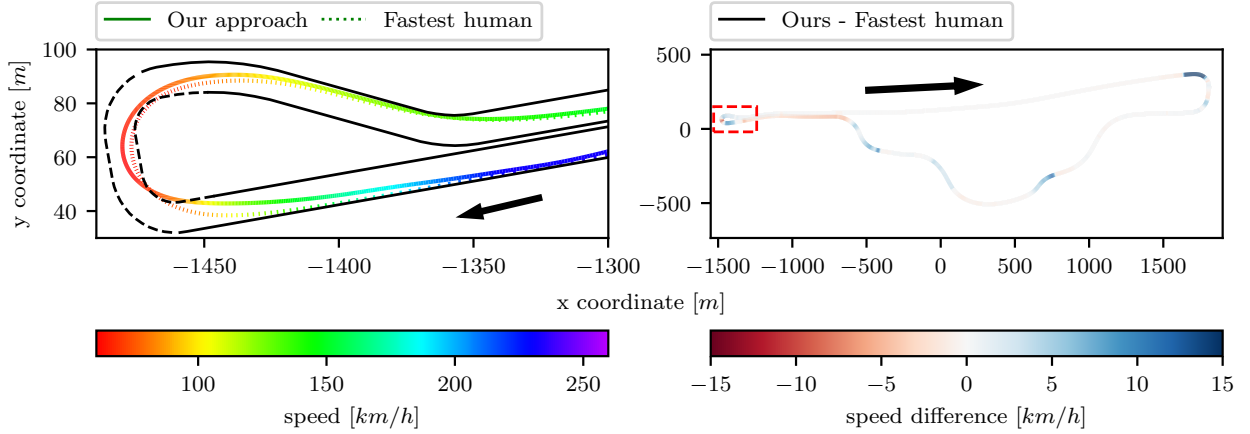


Fig. 9. **Left:** the trajectory driven by our approach and the fastest human trajectory in the hairpin segment in setting C. The solid track border is represented by walls on the track, while the dashed track border is simply drawn onto the track. To not get off the track, drivers are forced to brake early. **Right:** the speed difference between our approach and the fastest human over the whole track. Positive values indicate our approach driving faster in a segment.

unseen curve shapes. Since the agent gets stuck in the hairpin curve, it is not able to finish the track.

3) Increased tire friction: The agent is able to adjust to the change in friction by making corrections when deviating from the expected trajectory. In 2 curves, the agent overestimates the outward force, leading to brief contact with the inner edge of the curve. This leads to a loss of 0.1 seconds on the baseline. **Lowered tire friction:** The agent is able to make corrections to its trajectory to adjust to the change in dynamics and drive out-in-out paths for all but the 3 sharpest curves. In the sharpest curves, the agent takes a too aggressive path, based on the higher friction experienced in training, leading to contact with the outer edge at the end of the curve.

The two experiments indicate that our agent is able to make corrections to its path to a certain degree. However, for some track segments the corrections are not strong enough to handle the change in dynamics.

4) Noise in Observations: The bottom of Figure 10 shows the lap times in setting A for different degrees of uniform noise added to the agent's observations compared to the noise-free baseline performance. With up to 2% noise the agent drives trajectories close to our noise-free baseline, still placing in front of the fastest human. More noise in the agent's observations leads to jitter in its action signal, resulting in a speed loss. With up to 9% noise our agent is still able to drive on the track without any contact with the edge of the track.

5) Delayed Inference: The top of Figure 10 shows the lap times in setting A when adding delays of various degrees to the agent's inference compared to the non-delayed baseline performance. For delays up to 20 ms the agent performs close to the baseline, in some cases even slightly faster, outperforming the fastest human reference lap in each case. With a delay of 50 ms the agent loses 0.3 seconds on the baseline due to delay related deviations from the optimal line, but still achieves a lap time among the top 10 human drivers. With a delay of 100 ms our agent brakes too late before the sharpest curve of the track, leading to wall contact and a loss of 1.4 seconds compared to our baseline. With a delay of

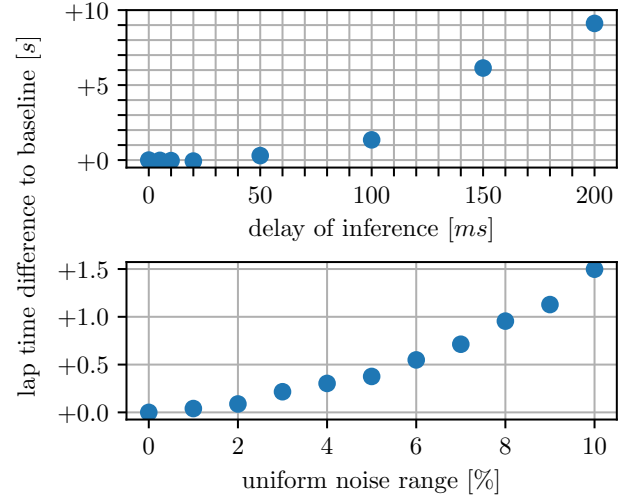


Fig. 10. **Top:** Loss in lap time when artificially delaying the agent's inference compared to undelayed baseline agent in setting A. **Bottom:** Loss in lap time when adding noise to the agent's observations.

150 ms our agent is no longer able to drive a competitive racing line, often colliding with the edge of the track and losing over 6 seconds on our baseline. It's possible that repeating the training process with the delay active is already sufficient to mitigate the decrease in performance. In addition, a predictive model such as the one proposed in [28] could be applied to counter the effect of the delay by estimating the real state based on the delayed state. In our comparison the human is disadvantaged by a relatively slow reaction time of approximately 0.2 seconds [29], compared to our agent, which is able to react in approximately 4 milliseconds, including both inference and communication with the PlayStation. We would argue, however, that this disadvantage is part of the human system and not part of the challenge of autonomous racing.

VI. CONCLUSION

In this paper, we have presented the first autonomous racing policy that achieves super-human performance in time trial

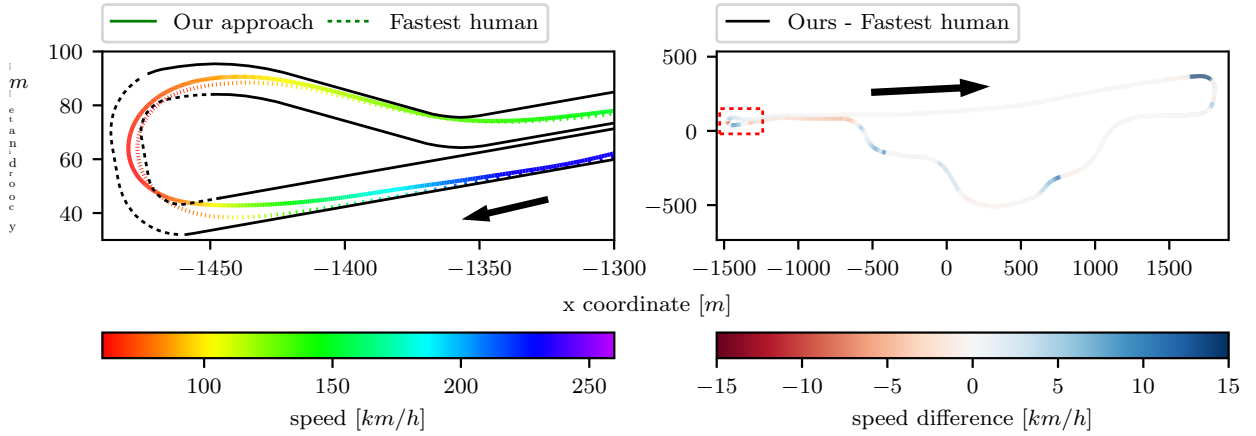


图 9. 左：我们的方法驱动的轨迹以及设置 C 中发夹段中最快的人类轨迹。实线轨道边界由轨道上的墙壁表示，而虚线轨道边界只是简单地绘制在轨道上。为了不离开赛道，车手被迫提前刹车。右图：我们的接近速度与整个赛道上最快的人类之间的速度差。正值表明我们的方法在某个细分市场中行使得更快。

看不见的曲线形状。由于智能体陷入发夹弯，因此无法完成赛道。

3) 增加轮胎摩擦力：当偏离预期轨迹时，智能体能够通过修正来适应摩擦力的变化。在 2 条曲线中，智能体高估了向外的力，导致与曲线内边缘短暂接触。这导致基线损失 0.1 秒。降低轮胎摩擦力：智能体能够修正其轨迹，以适应动态变化，并为除 3 个最急弯的弯道外的所有弯道驱动出由内向外的路径。在最急的弯道中，基于训练中经历的较高摩擦力，代理会采取过于激进的路径，导致与弯道末端的外边缘接触。

这两个实验表明我们的智能体能够在一定程度上对其路径进行修正。然而，对于某些赛道段，修正力度不足以应对动态变化。

4) 观察中的噪声：图 10 的底部显示了与无噪声基线性能相比，在设置 A 时添加到代理观察中的不同程度的均匀噪声的单圈时间。在噪声高达 2% 的情况下，代理驱动的轨迹接近我们的无噪声基线，但仍然领先于最快的人类。代理观察中的更多噪音会导致其动作信号抖动，从而导致速度损失。在噪音高达 9% 的情况下，我们的代理仍然能够在赛道上行驶，而不会与赛道边缘发生任何接触。

5) 延迟推理：图 10 的顶部显示了与非延迟基线性能相比，在设置 A 下向代理的推理添加不同程度的延迟时的单圈时间。对于高达 20 毫秒的延迟，代理的性能接近基线，在某些情况下甚至稍快一些，在每种情况下都优于最快的人类参考圈。在延迟 50 毫秒的情况下，由于与最佳线路的延迟相关的偏差，智能体在基线上损失了 0.3 秒，但仍然实现了前 10 名人类驾驶员中的单圈时间。由于延迟了 100 毫秒，我们的代理在赛道最急弯之前刹车太晚，导致与墙壁接触，与我们的基线相比损失了 1.4 秒。延迟为

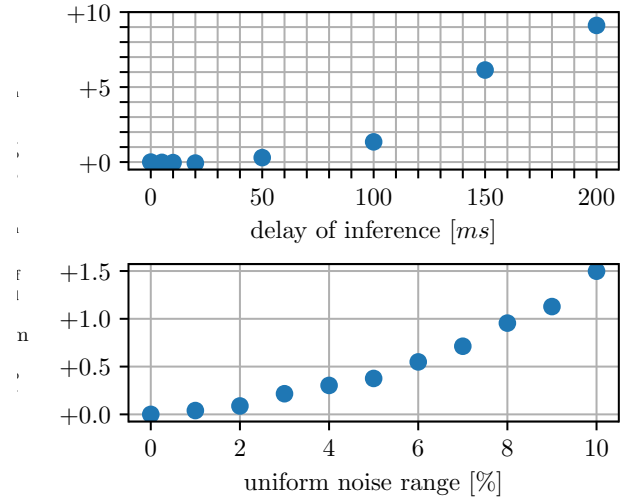


图 10. 上图：与设置 A 中未延迟的基线代理相比，人为延迟推理的推理时单圈时间的损失。底部：向代理的观察添加噪音时会损失单圈时间。

150 毫秒后，我们的代理不再能够驾驶竞技赛车线，经常与赛道边缘相撞，并在我们的基线上损失超过 6 秒。在延迟激活的情况下重复训练过程可能已经足以减轻性能的下降。此外，可以应用诸如[28]中提出的预测模型来通过基于延迟状态估计真实状态来抵消延迟的影响。在我们的比较中，人类的劣势是反应时间相对较慢，大约为 0.2 秒 [29]，而我们的代理能够在大约 4 毫秒内做出反应，包括推理和与 PlayStation 的通信。然而，我们认为这种劣势是人类系统的一部分，而不是自动驾驶赛车挑战的一部分。

六. 结论

在本文中，我们提出了第一个在计时赛中实现超人表现的自动驾驶赛车政策

settings in the racing simulator *Gran Turismo Sport*. The benefits of our approach are that it does not rely on human intervention, human expert data, or explicit path planning. It leads to trajectories that are qualitatively similar to those chosen by the best human players, while outperforming the best known human lap times in all three of our reference settings, including two different cars on two different tracks. This super-human performance has been achieved using limited computation power during both evaluation and training. Generating one control command with our policy network takes around 0.35 ms on a standard desktop PC CPU, and training, using 4 *PlayStation 4* game consoles and a single desktop PC, took less than 73 hours to achieve super-human performance.

Limitations of the present work include a) the restriction to single-player time trial races without other cars on the same track, and b) the constrained applicability of the learned control policies to a single track / car combination. We intend to address a) in future work by extending the observation space to allow the perception of other cars and modify the reward to disincentivize unfair driving behavior. To extend our approach to more track / car combinations we propose using more data-efficient RL algorithms, such as meta-RL, where the agent can adapt to new situations with only a small amount of new samples.

ACKNOWLEDGMENT

We are very grateful to Polyphony Digital Inc. for enabling this research. Furthermore, we would like to thank Kenta Kawamoto and Takuma Seno from Sony R&D Center Tokyo for their kind help and many fruitful discussions.

REFERENCES

- [1] G. Williams, P. Drews, B. Goldfain, J. M. Reh, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, May 2016, pp. 1433–1440.
- [2] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale rc cars," *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, 2015.
- [3] E. Frazzoli, M. A. Dahleh, and E. Feron, "Real-time motion planning for agile autonomous vehicles," *Journal of guidance, control, and dynamics*, vol. 25, no. 1, pp. 116–129, 2002.
- [4] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Reh, B. Boots, and E. A. Theodorou, "Information theoretic mpc for model-based reinforcement learning," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, May 2017, pp. 1714–1721.
- [5] G. Williams, P. Drews, B. Goldfain, J. M. Reh, and E. A. Theodorou, "Information-theoretic model predictive control: Theory and applications to autonomous driving," *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, 2018.
- [6] N. A. Spielberg, M. Brown, N. R. Kapania, J. C. Kegelman, and J. C. Gerdes, "Neural network vehicle models for high-performance automated driving," *Science Robotics*, vol. 4, no. 28, 2019.
- [7] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.
- [8] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, 2020.
- [9] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, "Control of a quadrotor with reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 2, no. 4, pp. 2096–2103, 2017.
- [10] A. Nagabandi, K. Konolige, S. Levine, and V. Kumar, "Deep dynamics models for learning dexterous manipulation," in *Conference on Robot Learning*. PMLR, 2020, pp. 1101–1112.
- [11] T. Novi, A. Liniger, R. Capitani, and C. Annicchiarico, "Real-time control for at-limit handling driving on a predefined path," *Vehicle System Dynamics*, vol. 58, no. 7, pp. 1007–1036, 2020.
- [12] U. Rosolia and F. Borrelli, "Learning how to autonomously race a car: a predictive control approach," *IEEE Transactions on Control Systems Technology*, 2019.
- [13] J. Kabzan, M. I. Valls, V. J. Reijgwart, H. F. Hendriks, C. Ehmke, M. Prajapat, A. Bühler, N. Gosala, M. Gupta, R. Sivanesan *et al.*, "Amz driverless: The full autonomous racing system," *Journal of Field Robotics*, vol. 37, no. 7, pp. 1267–1294, 2020.
- [14] C. J. Ostafew, A. P. Schoellig, and T. D. Barfoot, "Robust constrained learning-based nmpe enabling reliable mobile robot path tracking," *The International Journal of Robotics Research*, vol. 35, no. 13, pp. 1547–1563, 2016.
- [15] R. Verschuere, S. De Bruyne, M. Zanon, J. V. Frasch, and M. Diehl, "Towards time-optimal race car driving using nonlinear mpc in real-time," in *53rd IEEE conference on decision and control*. IEEE, 2014, pp. 2505–2510.
- [16] J. Kabzan, L. Hewing, A. Liniger, and M. N. Zeilinger, "Learning-based model predictive control for autonomous racing," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3363–3370, 2019.
- [17] D. A. Pomerleau, "Alvin: An autonomous land vehicle in a neural network," in *Advances in neural information processing systems*, 1989, pp. 305–313.
- [18] M. Bojarski, D. D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, "End to end learning for self-driving cars," *CoRR*, vol. abs/1604.07316, 2016.
- [19] Y. Pan, C.-A. Cheng, K. Saigol, K. Lee, X. Yan, E. Theodorou, and B. Boots, "Agile autonomous driving using end-to-end deep imitation learning," in *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, June 2018.
- [20] M. Jaritz, R. de Charette, M. Toromanoff, E. Perot, and F. Nashashibi, "End-to-end race driving with deep reinforcement learning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, May 2018, pp. 2070–2075.
- [21] M. Riedmiller, M. Montemerlo, and H. Dahlkamp, "Learning to drive a real car in 20 minutes," in *2007 Frontiers in the Convergence of Bioscience and Information Technologies*, Oct 2007, pp. 645–650.
- [22] A. Kendall, J. Hawke, D. Janz, P. Mazur, D. Reda, J. Allen, V. Lam, A. Bewley, and A. Shah, "Learning to drive in a day," in *2019 International Conference on Robotics and Automation (ICRA)*, May 2019, pp. 8248–8254.
- [23] S. Grigorescu, B. Tranea, T. Cocias, and G. Macesanu, "A survey of deep learning techniques for autonomous driving," *Journal of Field Robotics*, vol. 37, no. 3, pp. 362–386, 2020.
- [24] P. Cai, X. Mei, L. Tai, Y. Sun, and M. Liu, "High-speed autonomous drifting with deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1247–1254, 2020.
- [25] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning*, 2018, pp. 1856–1865.
- [26] G. Kahn, A. Villafior, V. Pong, P. Abbeel, and S. Levine, "Uncertainty-aware reinforcement learning for collision avoidance," *CoRR*, vol. abs/1702.01182, 2017.
- [27] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev *et al.*, "Grandmaster level in starcraft ii using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [28] V. Firoiu, T. Ju, and J. Tenenbaum, "At human speed: Deep reinforcement learning with action delay," *CoRR*, vol. abs/1810.07286, 2018.
- [29] E. A. C. Thomas, "Information theory of choice-reaction times. by d. r. j. laming," *British Journal of Mathematical and Statistical Psychology*, vol. 22, no. 1, pp. 103–104, 1969.
- [30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [31] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, 2011, pp. 627–635.
- [32] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse *et al.*, "Dota 2 with large scale deep reinforcement learning," *arXiv preprint arXiv:1912.06680*, 2019.

赛车模拟器 *Gran Turismo Sport* 中的设置。我们的方法的好处是它不依赖于人类干预、人类专家数据或明确的路径规划。它产生的轨迹在质量上与最优秀的人类玩家选择的轨迹相似，同时在我们的所有三个参考设置（包括两条不同赛道上的两辆不同的汽车）中超越了最著名的人类单圈时间。这种超人的表现是在评估和训练过程中使用有限的计算能力实现的。在标准台式电脑 CPU 上，使用我们的策略网络生成一个控制命令大约需要 0.35 毫秒，而使用 4 个 *PlayStation 4* 游戏机和一台台式电脑进行训练，只需要不到 73 小时即可实现超人性能。

目前工作的局限性包括a) 对同一赛道上没有其他汽车的单人计时赛的限制，以及b) 学习的控制策略对单个赛道/汽车组合的适用性受到限制。我们打算在未来的工作中解决a) 问题，通过扩展观察空间以允许其他车辆的感知并修改奖励以抑制不公平的驾驶行为。为了将我们的方法扩展到更多的赛道/汽车组合，我们建议使用数据效率更高的强化学习算法，例如元强化学习算法，其中代理只需少量新样本即可适应新情况。

致谢

我们非常感谢 Polyphony Digital Inc. 促成了这项研究。此外，我们还要感谢索尼东京研发中心的 Kenta Kawamoto 和 Takuma Seno 的友好帮助和许多富有成效的讨论。

参考

- [1] G. Williams, P. Drews, B. Goldfain, J. M. Reh, and E. A. Theodorou, “采用模型预测路径积分控制的激进驾驶”, *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016 年 5 月, 第 1433-1440 页。[2] A. Liniger, A. Domahidi 和 M. Morari, “基于优化的 1:43 比例遥控汽车自主赛车”, *Optimal Control Applications and Methods*, 卷. 36, 没有. 5, 第 628-647 页, 2015 年。[3] E. Frazzoli, M. A. Dahleh 和 E. Feron, “敏捷自动驾驶车辆的实时运动规划”, *Journal of guidance, control, and dynamics*, 卷. 25, 没有. 1, 第 116-129 页, 2002 年。[4] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Reh, B. Boots 和 E. A. Theodorou, “基于模型的强化学习的信息理论 mpc”, *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017 年 5 月, 第 1714-1721 页。[5] G. Williams, P. Drews, B. Goldfain, J. M. Reh 和 E. A. Theodorou, “信息理论模型预测控制: 自动驾驶的理论和应用”, *IEEE Transactions on Robotics*, 卷. 34, 没有. 6, 第 1603-1622 页, 2018 年。[6] N. A. Spielberg, M. Brown, N. R. Kapania, J. C. Kegelmann 和 J. C. Gerdes, “用于高性能自动驾驶的神经网络车辆模型”, *Science Robotics*, 卷. 4, 没有. 2019 年 2 月 28 日。[7] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun 和 M. Hutter, “学习腿式机器人的敏捷和动态运动技能”, *Science Robotics*, 卷. 4, 没有. 26, p. eaa5872, 2019。[8] J. Lee, J. Hwangbo, L. Weillhausen, V. Koltun 和 M. Hutter, “在具有挑战性的地形上学习四足运动”, *Science Robotics*, 卷. 5, 没有. 47, 2020。[9] J. Hwangbo, I. Sa, R. Siegwart 和 M. Hutter, “通过强化学习控制四旋翼飞行器”, *IEEE Robotics and Automation Letters*, 卷. 2, 没有. 4, 第 2096-2103 页, 2017 年。
- [10] A. Nagabandi, K. Konolige, S. Levine 和 V. Kumar, “用于学习灵巧操作的深度动力学模型”, 参见 *Conference on Robot Learning*. PM LR, 2020 年, 第 1101-1112 页。
- [11] T. Novi, A. Liniger, R. Capitani 和 C. Annicchiarico, “在预定路径上进行极限操控驾驶的实时控制”, *Vehicle System Dynamics*, 卷. 58, 没有. 7, 第 1007-1036 页, 2020 年。[12] U. Rosolia 和 F. Borrelli, “学习如何自动驾驶汽车: 预测控制方法”, *IEEE Transactions on Control Systems Technology*, 2019 年。[13] J. Kabzan, M. I. Valls, V. J. Reijgwart, H. F. Hendriks, C. Ehmke, M. Prajapat, A. Bühler, N. Gosala, M. Gupta, R. Sivanesan *et al.*, “Amz 无人驾驶: 完整的自主赛车系统”, *Journal of Field Robotics*, 卷. 37, 没有. 7, 第 1267-1294 页, 2020 年。[14] C. J. Ostafew, A. P. Schoellig 和 T. D. Barfoot, “基于鲁棒约束学习的 nmpc, 实现可靠的移动机器人路径跟踪”, *The International Journal of Robotics Research*, 卷. 35, 没有. 13, 第 1547-1563 页, 2016 年。[15] R. Verschuere, S. De Bruyne, M. Zanon, J. V. Frasch 和 M. Diehl, “实时使用非线性 MPC 实现时间最优赛车驾驶”, 见 *53rd IEEE conference on decision and control*. IEEE, 2014 年, 第 2505-2510 页。[16] J. Kabzan, L. Hewing, A. Liniger 和 M. N. Zeilinger, “基于学习的自动赛车模型预测控制”, *IEEE Robotics and Automation Letters*, 卷. 4, 没有. 4, 第 3363-3370 页, 2019。[17] D. A. Pomerleau, “Alvin: 神经网络中的自主陆地车辆”, *Advances in neural information processing systems*, 1989 年, 第 305-313 页。[18] M. Bojarski, D. D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao 和 K. Zieba, “自动驾驶汽车的端到端学习”, *CoRR*, 卷. abs/1604.07316, 2016。[19] Y. Pan, C.-A. Cheng, K. Saigol, K. Lee, X. Yan, E. Theodorou 和 B. Boots, “使用端到端深度模仿学习的敏捷自动驾驶”, 发表于 *Proceedings of Robotics: Science and Systems*, 宾夕法尼亚州匹兹堡, 2018 年 6 月。[20] M. Jaritz, R. de Charette, M. Toromanoff, E. Perot 和 F. Nashashibi, “利用深度强化学习的端到端赛车驾驶”, 载于 *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018 年 5 月, 第 2070-2075 页。[21] M. Riedmiller, M. Montemero 和 H. Dahlkamp, “在 20 分钟内学会驾驶一辆真正的汽车”, 载于 *2007 Frontiers in the Convergence of Bioscience and Information Technologies*, 2007 年 10 月, 第 645-650 页。[22] A. Kendall, J. Hawke, D. Janz, P. Mazur, D. Reda, J. Allen, V. Lamm, A. Bewley 和 A. Shah, “一天学会开车”, 载于 *2019 International Conference on Robotics and Automation (ICRA)*, 2019 年 5 月, 第 8248-8254 页。[23] S. Grigorescu, B. Trasnea, T. Cocias 和 G. Macesanu, “自动驾驶深度学习技术调查”, *Journal of Field Robotics*, 卷. 37, 没有. 3, pp. 362-386, 2020。[24] P. Cai, X. Mei, L. Tai, Y. Sun, and M. Liu, “深度强化学习的高速自主漂移”, *IEEE Robotics and Automation Letters*, vol. 3, pp. 362-386, 2020. 5, 没有. 2, 第 1247-1254 页, 2020 年。[25] T. Haarnoja, A. Zhou, P. Abbeel 和 S. Levine, “软演员批评家: 随机演员的非策略最大熵深度强化学习”, *International Conference on Machine Learning*, 2018 年, 第 1856-1865 页。[26] G. Kahn, A. Villafior, V. Pong, P. Abbeel 和 S. Levine, “用于避免碰撞的不确定性感知强化学习”, *CoRR*, 卷. abs/1702.01182, 2017。[27] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev *et al.*, “星际争霸 2 中使用多智能体强化学习的大师级别”, *Nature*, 卷. 575, 没有. 7782, 第 350-354 页, 2019。[28] V. Firoiu, T. Ju 和 J. Tenenbaum, “以人类速度: 具有动作延迟的深度强化学习”, *CoRR*, 卷. abs/1810.07286, 2018。[29] E.A.C. Thomas, “选择反应时间的信息论。作者: d.r.j.laming”, *British Journal of Mathematical and Statistical Psychology*, 卷. 22, 没有. 1, 第 103-104 页, 1969 年。[30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford 和 O. Klimov, “近端策略优化算法”, *arXiv preprint arXiv:1707.06347*, 2017 年。[31] S. Ross, G. Gordon 和 D. Bagnell, “将模仿学习和结构化预测减少到无悔在线学习”, 载于 *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, 2011 年, 第 627-635 页。[32] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse *et al.*, “大规模深度强化学习的 Dota 2”, *arXiv preprint arXiv:1912.06680*, 2019 年。

- [33] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of go with deep neural networks and tree search,” *Nature*, vol. 529, pp. 484–503, 2016. [Online]. Available: <http://www.nature.com/nature/journal/v529/n7587/full/nature16961.html>
- [34] G. Barth-Maron, M. W. Hoffman, D. Budden, W. Dabney, D. Horgan, D. TB, A. Muldal, N. Heess, and T. Lillicrap, “Distributional policy gradients,” in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=SyZipzbCb>
- [35] T. Hellstrom and O. Ringdahl, “Follow the past: a path-tracking algorithm for autonomous vehicles,” *International journal of vehicle autonomous systems*, vol. 4, no. 2-4, pp. 216–224, 2006.
- [36] P. Ritzer, C. Winter, and J. Brembeck, “Advanced path following control of an overactuated robotic vehicle,” in *2015 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2015, pp. 1120–1125.
- [37] J. Ni, J. Hu, and C. Xiang, “Robust path following control at driving/handling limits of an autonomous electric racecar,” *IEEE Transactions on Vehicular Technology*, vol. 68, no. 6, pp. 5518–5526, 2019.

[33] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel 和 D. Hassabis, “利用深度神经网络和树搜索掌握围棋游戏”, *Nature*, 卷。529, 第 484–503 页, 2016 年。[在线]。网址: <http://www.nature.com/nature/journal/v529/n7587/full/nature16961.html> [34] G. Barth-Maron, M. W. Hoffman, D. Budden, W. Dabney, D. Horgan, D. TB, A. Muldal, N. Heess 和 T. Lillicrap, “分配政策梯度”, 载于 *International Conference on Learning Representations*, 2018。[在线]。可用: <https://openreview.net/forum?id=SyZipzbCb> [35] T. Hellstrom 和 O. Ringdahl, “追随过去: 自动驾驶汽车的路径跟踪算法”, *International journal of vehicle autonomous systems*, 卷。4, 没有。2-4, 第 216–224 页, 2006 年。[36] P. Ritzer, C. Winter 和 J. Brembeck, “过驱动机器人车辆的高级路径跟踪控制”, 参见 *2015 IEEE Intelligent Vehicles Symposium (IV)*。IEEE, 2015 年, 第 1120–1125 页。[37] J. Ni, J. Hu, and C. Xiang, “自主电动赛车驾驶/操控极限下的鲁棒路径跟踪控制”, *IEEE Transactions on Vehicular Technology*, 卷。68, 没有。6, 第 5518–5526 页, 2019 年。

APPENDIX

A. Prior Approaches

At an early stage of this project, we have used different learning methods, including proximal policy optimization (PPO) [30] and imitation learning. Both approaches didn't come close to the performance we have achieved with SAC. In particular, PPO required much more training data and suffered from premature convergences due to its state-independent exploration. The imitation learning method suffered from the DAgger problem [31] and its performance is upper bound by the training data, which was collected from human players (top 1% driver trajectories). We prefer to not include them due to their poor performance but rather focus more on other interesting findings, such as comparison to human players.

B. GTS Access

The GTS simulator runs on a *PlayStation 4*, while our agent runs on a separate desktop computer. We do not have direct access to the GTS simulator, neither do we have insights into the car dynamics modeled by GTS. Instead, we interact with GTS over a dedicated API via ethernet connection. The API provides the current state of up to 20 simulated cars and accepts car control commands, which are active until the next command is received. Whereas previous work in the area of RL often ran thousands of simulations in parallel to collect training data [9], [32], [33], we can only run one simulation per *PlayStation*, and for this work we make use of only 4 *PlayStations* during training. Moreover, the simulation runs in real-time and cannot be sped up for data collection during training nor can it be paused to create additional time for decision making. The simulator's state is updated at a frequency of 60 Hz, but, to reduce the load on the *PlayStations* when controlling 20 cars, we limit the command frequency to 10 Hz during training. During evaluation, we switch to one car, increasing the agent's action frequency to 60 Hz. We use a standard desktop computer with a *i7-8700* processor with a clock speed of 3.20GHz for inference and a *GeForce GTX 1080 Ti* graphics card for backpropagating through the models.

C. Network Training

To train the networks we roll out 4x20 trajectories per epoch in parallel, with a length of 100 seconds each, by using all 20 cars available on each of the 4 *Playstations*. To minimize disturbance between cars during training, we initialize the position of agents equally distributed over the racing track with an initial speed of 100 km/h, which we found can accelerate training, since it allows the agents to faster approach the maximal feasible segment speeds, which are of interest for finding the fastest possible trajectory.

For the training process we make use of the TensorFlow-based SAC implementation by OpenAI². We modified the code base to allow to asynchronously learn during roll outs and changed the default 1-step TD error to a 5-step TD error to stabilize training as also used in [34]. The combined training

of all 4 networks takes approximately 35 seconds per epoch, which leaves the total epoch time at 100 seconds. Because of the real-time character of the GTS environment, an extensive hyperparameter search is not feasible. We therefore adapt most default hyperparameters from the OpenAI implementation, except for those listed in table II.

Hyperparameter	Value
Mini-batch size	4,096
Replay buffer size	4×10^6
Learning rate	3×10^{-4}
Update steps per epoch	5,120
Reward scale ($1/\alpha$)	100
Exponential discount (γ) for the "Mazda Demio"	0.98
Exponential discount (γ) for the "Audi TT Cup"	0.982
Wall contact penalty scale (c_w)	5×10^{-4}
Number of range finders (M)	13 (every 15°)
Number of curvature measurements (N)	10 (every 0.2s)

TABLE II

HYPERPARAMETERS USED FOR THE SAC ALGORITHM AND THE GTS SIMULATION. DENSER RANGE FINDERS LEAD TO SLOWER CONVERGENCE BUT SHOWED NO ADDITIONAL IMPROVEMENTS IN THE LEARNED POLICY.

D. Curvature construction

We base the input feature selection on the results of a pre-study on GTS conducted with behavioral cloning, aiming to learn human-like driving based on a regression of expert's actions on visited states. We iterated over combinations of subsets of features provided by GTS as well as additionally constructed features, which lead to the following feature selection: 1) The linear velocity $\mathbf{v}_t \in \mathbb{R}^3$ and the linear acceleration $\dot{\mathbf{v}}_t \in \mathbb{R}^3$. 2) The Euler angle $\theta_t \in (-\pi, \pi]$ between the 2D vector that defines the agent's rotation in the horizontal plane and the unit tangent vector that is tangent to the centerline at the projection point. This angle is the only direct way for the policy network to detect if a car is facing the wrong direction. 3) Distance measurements $\mathbf{d}_t \in \mathbb{R}^M$ of M rangefinders that measure the distance from the vehicle's center point to the M edge points of its surrounding objects, such as the edge of the race track. The rangefinders are equally distributed in the front 180° of the car's view. 4) The previous steering command δ_{t-1} . 5) A binary flag with $w_t = 1$ indicating wall contact and 6) N sampled curvature measurement of the course centerline in the near future $\mathbf{c}_t \in \mathbb{R}^N$. The curvature is represented through an interpolation of the inverse radii of circles fitted through centerline points provided by GTS. We therefore represent the observation at a given time step t as a vector denoted as $\mathbf{s}_t = [\mathbf{v}_t, \dot{\mathbf{v}}_t, \theta_t, \mathbf{d}_t, \delta_{t-1}, w_t, \mathbf{c}_t]$. To allow for a fair comparison with human drivers, we only use features that humans can either directly perceive or deduce from the GTS simulation.

The curvature is represented through an interpolation of the inverse radii of circles fitted through centerline points provided by GTS, as can be seen in Figure 11. They are equally distributed from 1.0 to 2.8 seconds into the future from the car's current position, estimated with the car's current speed.

²github.com/openai/spinningup

附录

A. Prior Approaches

在该项目的早期阶段，我们使用了不同的学习方法，包括近端策略优化（PPO）[30]和模仿学习。这两种方法都无法达到我们通过 SAC 实现的性能。特别是，PPO 需要更多的训练数据，并且由于其与状态无关的探索而遭受过早收敛。模仿学习方法遇到了 DAgger 问题 [31]，其性能受到训练数据的上限，训练数据是从人类玩家（前 1% 的驾驶员轨迹）收集的。由于他们的表现不佳，我们宁愿不将他们包括在内，而是更多地关注其他有趣的发现，例如与人类玩家的比较。

B. GTS Access

GTS 模拟器在 PlayStation 4 上运行，而我们的代理在单独的台式计算机上运行。我们无法直接访问 GTS 模拟器，也无法深入了解 GTS 建模的汽车动力学。相反，我们通过以太网连接通过专用 API 与 GTS 进行交互。API 提供最多 20 辆模拟汽车的当前状态并接受汽车控制命令，这些命令在收到下一个命令之前一直有效。尽管之前 RL 领域的工作经常并行运行数千次模拟来收集训练数据 [9]、[32]、[33]，但我们只能对每个 PlayStation 运行一次模拟，并且对于这项工作，我们在训练期间仅使用 4 个 PlayStations。此外，模拟是实时运行的，不能在训练期间加速数据收集，也不能暂停模拟以创造额外的决策时间。模拟器的状态以 60 Hz 的频率更新，但是，为了减少控制 20 辆汽车时 PlayStations 上的负载，我们在训练期间将命令频率限制为 10 Hz。在评估过程中，我们切换到一辆车，将智能体的动作频率增加到 60 Hz。我们使用配备时钟速度为 3.20GHz 的 i7-8700 处理器的标准台式计算机进行推理，并使用 GeForce GTX 1080 Ti 显卡通过模型进行反向传播。

C. Network Training

为了训练网络，我们通过使用 4 个 Playstations 中每一个上可用的所有 20 辆汽车，并行推出每个 epoch 的 4x20 轨迹，每个轨迹长度为 100 秒。为了最大限度地减少训练期间汽车之间的干扰，我们以 100 km/h 的初始速度初始化为赛道上均匀分布的智能体的位置，我们发现这可以加速训练，因为它允许智能体更快地接近最大可行的路段速度，这对于寻找最快的可能轨迹很重要。

在训练过程中，我们使用 OpenAI² 基于 TensorFlow 的 SAC 实现。我们修改了代码库以允许在推出期间异步学习，并将默认的 1 步 TD 误差更改为 5 步 TD 误差以稳定训练，这也在 [34] 中使用。联合训练

所有 4 个网络的每个 epoch 大约需要 35 秒，这使得总 epoch 时间为 100 秒。由于 GTS 环境的实时特性，广泛的超参数搜索是不可行的。因此，除了表 II 中列出的参数外，我们采用了 OpenAI 实现中的大多数默认超参数。

Hyperparameter	Value
Mini-batch size	4,096
Replay buffer size	4×10^6
Learning rate	3×10^{-4}
Update steps per epoch	5,120
Reward scale ($1/\alpha$)	100
Exponential discount (γ) for the “Mazda Demio”	0.98
Exponential discount (γ) for the “Audi TT Cup”	0.982
Wall contact penalty scale (c_w)	5×10^{-4}
Number of range finders (M)	13 (every 15°)
Number of curvature measurements (N)	10 (every 0.2s)

TABLE II

用于 SAC 算法和 GTS 模拟的超参数。更密集的测距仪会导致收敛速度变慢，但在学习策略方面没有显示出任何额外的改进。

D. Curvature construction

我们的输入特征选择基于通过行为克隆进行的 GTS 预研究的结果，旨在基于专家对访问状态的行为的回归来学习类人驾驶。我们迭代 GTS 提供的特征子集以及附加构造的特征的组合，从而导致以下特征选择：1) 线速度 $\mathbf{v}_t \in \mathbb{R}^3$ 和线加速度 $\dot{\mathbf{v}}_t \in \mathbb{R}^3$ 。2) 定义智能体在水平面内旋转的 2D 向量与与投影点中心线相切的单位切向量之间的欧拉角 $\theta_t \in (-\pi, \pi]$ 。这个角度是策略网络检测汽车是否面向错误方向的唯一直接方式。3) M 测距仪的距离测量 $\mathbf{d}_t \in \mathbb{R}^M$ ，用于测量从车辆中心点到其周围物体的 M 边缘点（例如赛道边缘）的距离。测距仪均匀分布在汽车视野前方 180° 范围内。4) 先前的转向命令 δ_{t-1} 。5) 一个二元标志，其中 $w_t = 1$ 指示墙壁接触，6) N 在不久的将来 $\mathbf{c}_t \in \mathbb{R}^N$ 采样球场中心线的曲率测量。曲率是通过 GTS 提供的中心线点拟合的圆的倒半径插值来表示的。因此，我们将给定时间步长 t 的观测值表示为 $\mathbf{s}_t = [\mathbf{v}_t, \dot{\mathbf{v}}_t, \theta_t, \mathbf{d}_t, \delta_{t-1}, w_t, \mathbf{c}_t]$ 的向量。为了与人类驾驶员进行公平比较，我们仅使用人类可以直接感知或从 GTS 模拟中推断出的特征。

曲率通过 GTS 提供的中心线点拟合的圆的倒数半径插值来表示，如图 11 所示。它们从汽车当前位置到未来 1.0 到 2.8 秒的时间内均匀分布，并根据汽车的当前速度进行估计。

²github.com/openai/spinningup

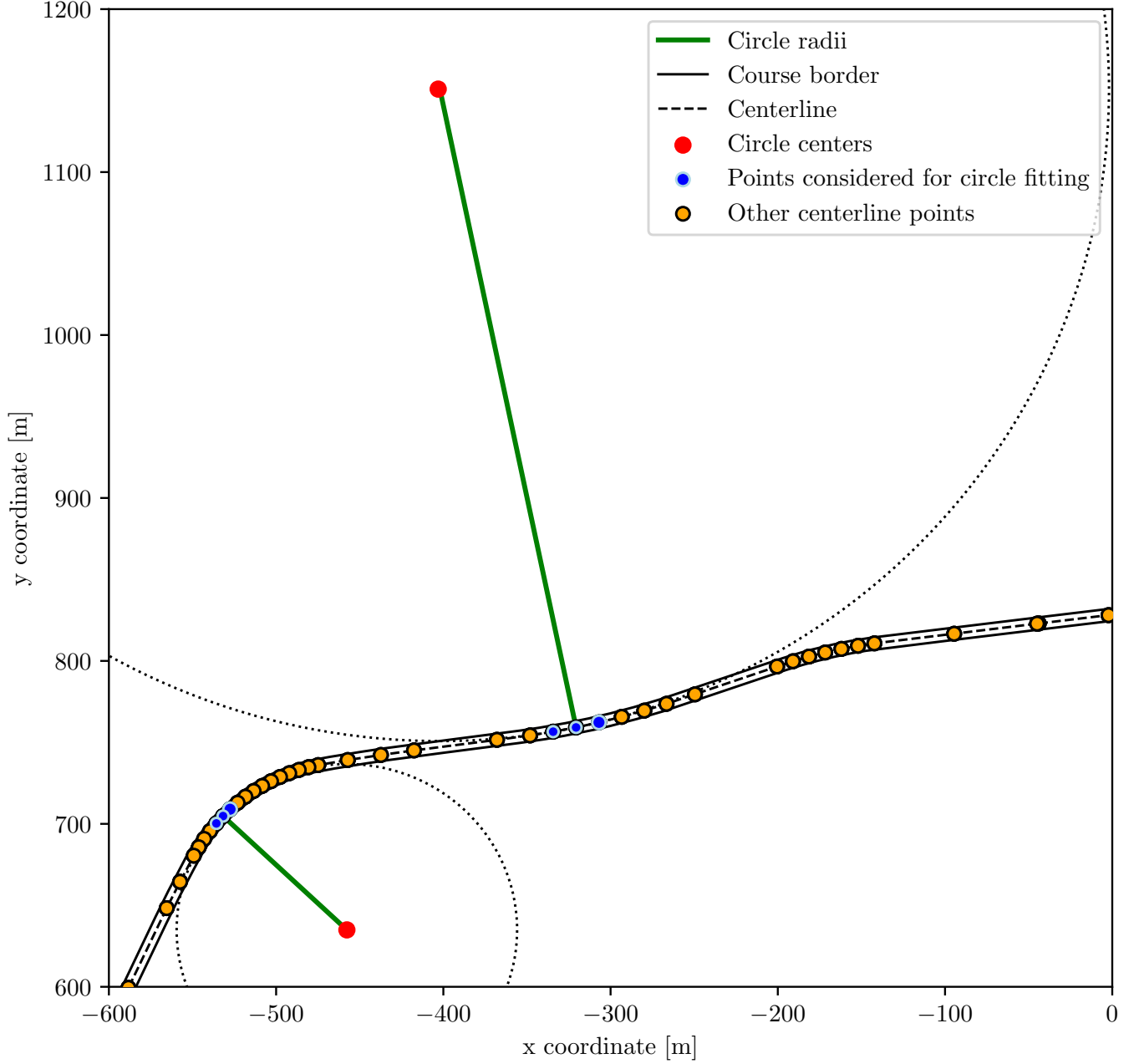


Fig. 11. **Construction of the curvature measurement.** To represent the curvature of the centerline, we make use of centerline points provided by the GTS simulation. The points are distributed such that regions with higher curvature are represented by more points. This allows getting a precise curvature measurement from looking at only three neighboring points. We make use of that property by defining the curvature for each centerline point by the inverse radius of the circle given by that point and its two neighbors. Right curvature is represented by negative inverted radii, left curvature by positive inverted radii. We then interpolate between centerline points to end up with a continuous representation of the curvature over the course progress.

E. Control Signal

We define the action vector $\mathbf{a}_t = [\delta_t, \omega_t]$ by two control signals: a steering angle $\delta_t \in [-\pi/6, \pi/6](rad)$ corresponding to the angle of the car's front wheels and a combined throttle-brake signal $\omega_t \in [-1, 1]$, where $\omega > 0$ denotes throttle and $\omega \leq 0$ represents braking. The magnitude of the throttle or the brake is proportional to the absolute value $\|\omega\|$. For example, $\omega = 1$ is 100% throttle, $\omega = -1$ is 100% brake, and $\omega = 0$ is no throttle and no brake. Combining the two signals reduces the complexity of the task. Since human expert

recordings show that the best human strategies do not involve the simultaneous use of throttle and brake, we expect not to significantly restrict the agent by combining the two signals. To limit the outputs of the policy network to the valid range, we apply a *tanh* activation to the output layer as proposed in [25].

We use the automatic gearshift provided by GTS, since the used API does currently not allow to manually change gears. This option is also available to human players, but experienced players mostly use manual gearshift to have more control of

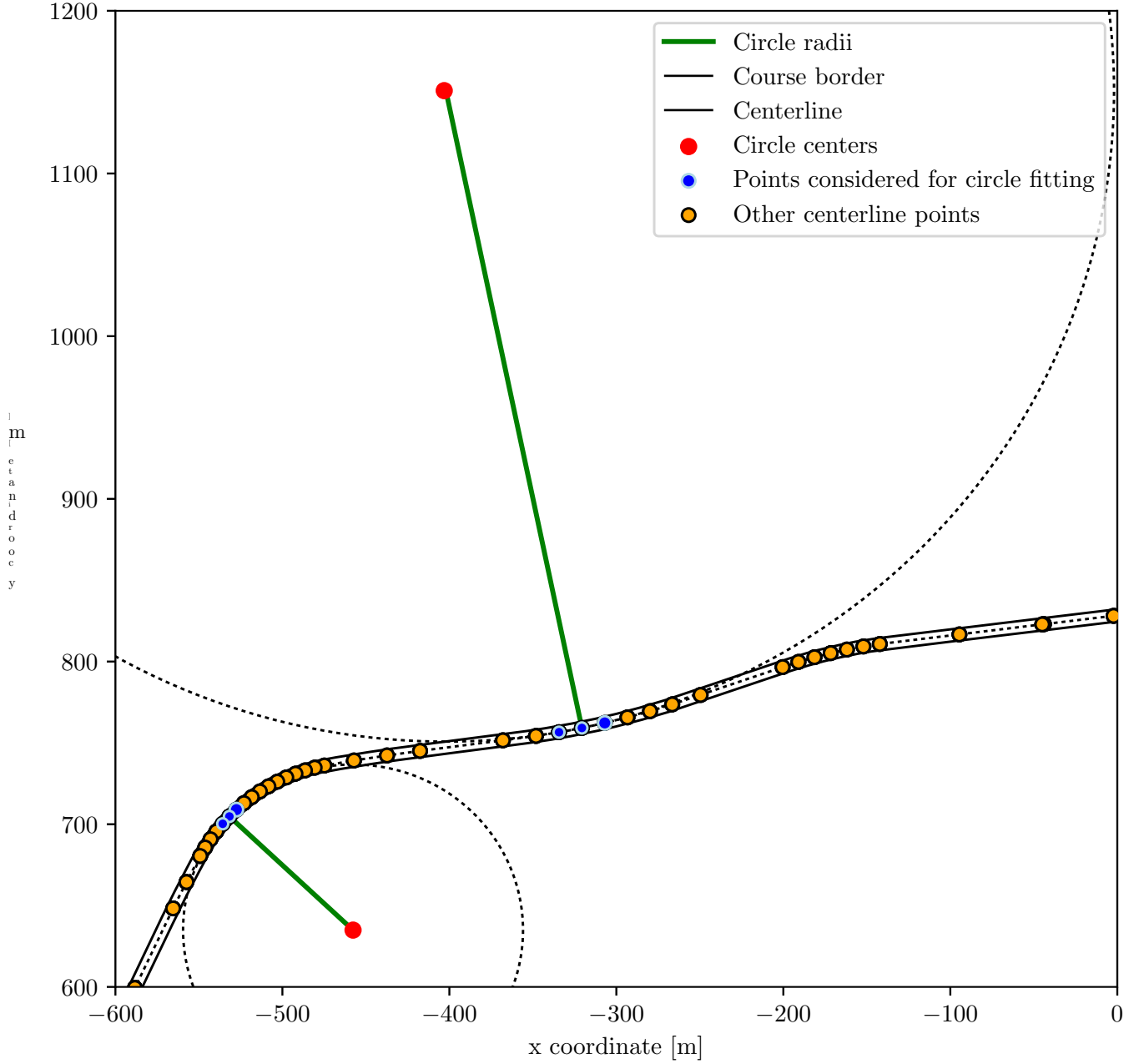


图 11. 曲率测量的构造。为了表示中心线的曲率，我们利用 GTS 模拟提供的中心线点。这些点的分布使得曲率较高的区域由更多的点表示。这样可以通过仅查看三个相邻点来获得精确的曲率测量。我们利用该属性，通过由该点及其两个邻居给出的圆的半径的倒数来定义每个中心线点的曲率。右曲率由负倒半径表示，左曲率由正倒半径表示。然后，我们在中心线点之间进行插值，最终得到整个过程中曲率的连续表示。

E. Control Signal

我们通过两个控制信号定义动作向量 $\mathbf{a}_t = [\delta_t, \omega_t]$ ：对应于汽车前轮角度的转向角 $\delta_t \in [-\pi/6, \pi/6](rad)$ 和组合油门制动信号 $\omega_t \in [-1, 1]$ ，其中 $\omega > 0$ 表示油门， $\omega \leq 0$ 表示制动。节流或中断的幅度与绝对值 $\|\omega\|$ 成正比。例如， $\omega = 1$ 为 100% 油门， $\omega = -1$ 为 100% 刹车， $\omega = 0$ 为无油门且无刹车。组合两个信号可以降低任务的复杂性。既然人类专家

记录表明，最好的人类策略不涉及同时使用油门和刹车，我们预计不会通过组合这两个信号来显着限制代理。为了将策略网络的输出限制在有效范围内，我们将 $\tanh$ 激活应用于输出层，如[25]中提出的。

我们使用GTS提供的自动换挡，因为所使用的API目前不允许手动换挡。人类玩家也可以使用此选项，但经验丰富的玩家大多使用手动变速来更好地控制

when to shift.

F. Analysis of the Results by a Domain Expert

To improve our understanding of the achieved results, we invited *Gran Turismo* domain expert TG (name omitted for reasons of anonymity), who has achieved top performance in several national and international competitions, to race in our reference settings and compare his performance against our approach. TG competed in two of our three reference settings and achieved lap times in the top 0.36 and 0.23 percentile of our human reference data.

When asked for his opinion on the policies' driving style, TG stated:

"The policy drives very aggressively, but I think this is only possible through its precise actions. I could technically also drive the same trajectory, but in 999 out of a 1000 cases, me trying that trajectory results in wall contact which destroys my whole lap time and I would have to start a new lap from scratch."

G. Improvements over the built-in AI

In the 3 introduced reference settings the currently built-in AI is outperformed by a majority of human players as can be seen in Table I of the main manuscript. The built-in AI follows a pre-defined trajectory using a rule-based tracking approach, similar to other trajectory following approaches that have been widely studied in the control community [35]–[37]. Due to the non-linearity of the GTS dynamics, a slight deviation from such a trajectory strongly changes the new optimal trajectory, which makes it practically impossible to pre-compute and then track an optimal trajectory. The built-in AI solves this problem by using a cautious reference trajectory which includes a margin to the track's borders and allows recovery when deviating from the trajectory, as can be seen in Figure 8 of the main manuscript. This helps reducing the risk of contacting the track's side walls when deviating from the trajectory. However, it also leads to the trajectory's curves having smaller radii, forcing the AI to decelerate to not lose traction, resulting in curve exit speeds up to 50 km/h slower than those of our approach and the fastest human. The bottom of Figure 8 of the main manuscript shows how the built-in AI brakes in segments where neither our approach nor the fastest human brake, to be able to follow its reference trajectory. By learning a flexible driving policy without any explicit restrictions, our approach was able to overcome the shortcomings of the previous built-in AI and learn a driving policy more similar to that of a human expert driver regarding speed and path.

H. Reproducibility of experiments after GTS Update 1.57

Starting with [GTS version 1.57](#) released on April 23rd 2020, the simulation dynamics for the "Audi TT Cup '16" changed such that the achievable times for setting A and C of this paper are different. Under the new update a fair comparison to the human data sets of setting A and C, which were collected under an older GTS version, is therefore no longer possible.

TABLE III
TIME TRIAL COMPARISONS BETWEEN OUR APPROACH, HUMAN ONLINE COMPETITORS, AND THE BUILT-IN GTS AI FOR THE NEW SETTING A UNDER GTS VERSION 1.57.

Driver	Metric	Updated setting A
Ours	Lap time [min]	01:14.686
Human players	Fastest lap [min]	01:14.775
	Median lap [min]	01:21.794
	# participants	71,005
Built-in GTS AI	Lap time [min]	01:26.356
	Slower than x% of humans drivers	82.5%

For future comparison with this work we recommend the following human data set as a replacement for setting A: [kudosprime.com](#). The simulation settings used for this race are equal to those used in setting A of this paper, except that they're using the updated car dynamics from version 1.57+ as well as updated tires (RM instead of RH). The approach described in this paper achieves a lap time of 74.686 seconds after 5 days of training when applied on this new setting, as shown in Table III.

何时转移。

F. Analysis of the Results by a Domain Expert

为了加深我们对所取得结果的理解，我们邀请了在多项国内和国际比赛中取得顶尖表现的 *Gran Turismo* 领域专家 TG（因匿名原因省略姓名）在我们的参考设置中进行比赛，并将他的表现与我们的方法进行比较。TG 在我们的三个参考设置中的两个中进行了比赛，并在我们的人类参考数据中取得了前 0.36 和 0.23 个百分点的单圈时间。

当被问及对政策驾驶风格的看法时，TG 表示：

“The policy drives very aggressively, but I think this is only possible through its precise actions. I could technically also drive the same trajectory, but in 999 out of a 1000 cases, me trying that trajectory results in wall contact which destroys my whole lap time and I would have to start a new lap from scratch.”

G. Improvements over the built-in AI

在引入的 3 个参考设置中，当前内置 AI 的表现优于大多数人类玩家，如主手稿的表 I 所示。内置人工智能使用基于规则的跟踪方法遵循预先定义的轨迹，类似于控制界广泛研究的其他轨迹跟踪方法[35]-[37]。由于 GTS 动力学的非线性，与该轨迹的轻微偏差会强烈改变新的最优轨迹，这使得预先计算然后跟踪最优轨迹实际上是不可能的。内置人工智能通过使用谨慎的参考轨迹解决了这个问题，其中包括轨迹边界的余量，并允许在偏离轨迹时进行恢复，如主手稿的图 8 所示。这有助于降低偏离轨迹时接触轨道侧壁的风险。然而，这也导致轨迹的曲线半径更小，迫使 AI 减速以免失去牵引力，导致曲线出口速度比我们的方法和最快的人类慢 50 公里/小时。主手稿图 8 的底部显示了内置人工智能如何在既不是我们的接近也不是最快的人类制动的路段中制动，以便能够遵循其参考轨迹。通过学习没有任何明确限制的灵活驾驶策略，我们的方法能够克服以前内置人工智能的缺点，并学习在速度和路径方面更类似于人类专家驾驶员的驾驶策略。

H. Reproducibility of experiments after GTS Update 1.57

从 2020 年 4 月 23 日发布的 GTS 1.57 版本开始，“奥迪 TT Cup ’16”的模拟动态发生了变化，使得本文设置 A 和 C 的可实现时间不同。因此，在新的更新中，不再可能与旧 GTS 版本下收集的设置 A 和 C 的人类数据集进行公平比较。

表 III 我们的方法、人类在线竞争对手以及 GTS 版本 1.57 下新设置 A 的内置 GTS AI 之间的计时赛比较。

Driver	Metric	Updated setting A
Ours	Lap time [min]	01:14.686
Human players	Fastest lap [min]	01:14.775
	Median lap [min]	01:21.794
	# participants	71,005
Built-in GTS AI	Lap time [min]	01:26.356
	Slower than x% of humans drivers	82.5%

为了将来与这项工作进行比较，我们建议使用以下人类数据集作为设置 A 的替代：kudosprime.com。本场比赛使用的模拟设置与本文设置 A 中使用的模拟设置相同，不同之处在于它们使用的是 1.57+ 版本中更新的汽车动力学以及更新的轮胎（RM 而不是 RH）。当应用于这种新设置时，本文描述的方法在训练 5 天后实现了 74.686 秒的单圈时间，如表 III 所示。